
Informe Taller de Investigación Dirigida

**Búsqueda de algoritmos y herramientas para desarrollar y
resolver un juego en desarrollo**

Primer Semestre 2026

Estudiante: Alonso Tamayo

Profesores: Pedro Montealegre
Martín Ríos-Wilson

Fecha: 8 de julio de 2026

Índice

1. Introducción del Juego	2
2. Motivación de la Investigación	5
3. Marco Teórico	6
4. Resultados	12
4.1. Implementación del Algoritmo Genético	19
4.2. Resultados del Algoritmo Genético Configurado Manualmente	34
4.3. Análisis de Sensibilidad	36
4.3.1. Definición e Implementación	36
4.3.2. Resultados	38
4.4. Resultados del Algoritmo Genético Modificado	43
5. Discusión	45
5.1 Proyecciones a Futuro	46
5.1.1 Otros Algoritmos y Métodos	47
6. Bibliografía	50
7. Anexos	51

1. Introducción del Juego

El juego que se buscó resolver es uno de origen propio (A.1), y consiste en una serie de puzles en los que se pueden mover tanto el o los *jugadores* como los espacios en el nivel en los que estos pueden existir. Cada nivel existe en un plano cuadrículado de dos dimensiones de algún tamaño $N \times M$, y en cada posición existe una *casilla*, sobre la cual puede o no existir una *habitación*. Una *habitación* puede o no tener cualquier combinación de *conexiones*, que pueden, de manera independiente, apuntar a una *casilla* cardinalmente adyacente (arriba, abajo, izquierda o derecha); además, una *habitación* puede o no contener una *entidad*, que puede ser un *jugador* o una *salida*, entre otras más. El objetivo del juego es llevar al *jugador* a una *habitación* que contenga una *salida*, teniendo como regla que solo se permite el movimiento del *jugador* entre *habitaciones* si y solo si ambas tienen *conexiones* apuntándose entre sí, y no existe una *pared* entre medio, de esta manera, un nivel trivial de tamaño 3×1 podría verse así:

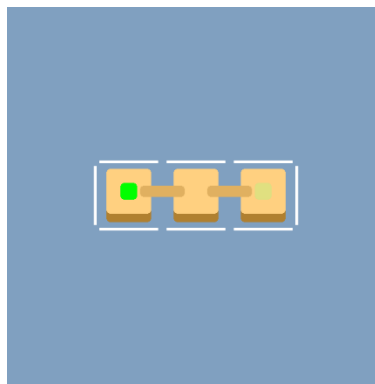


Figura 1: Nivel trivial de tamaño 3×1 con *paredes* (rectángulos blancos) delimitando sus bordes.

Muestra a un *jugador* (cuadrado verde) en la *habitación* izquierda y una *salida* (cuadrado verde transparente) en la derecha. (Fuente: Elaboración propia)

Otra *entidad* que puede contener una *habitación* es la *llave*, que un *jugador* puede tomar y guardar en su propio inventario si este entra en la *habitación* que la contenga, para luego poder consumirla al intentar moverse a través de una *conexión* bloqueada, dejando esta permanentemente abierta para poder volver a atravesarla en el futuro (cabe destacar que, si ambas *conexiones* están bloqueadas, un *jugador* necesitará consumir dos *llaves* para poder moverse a la otra *habitación*):

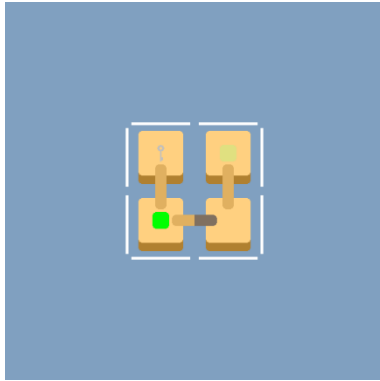


Figura 2: Nivel de tamaño 2×2 con una *llave* (en la *habitación* arriba-izquierda) y una *conexión* bloqueada (en gris oscuro, perteneciente a la *habitación* abajo-derecha).
(Fuente: Elaboración propia)

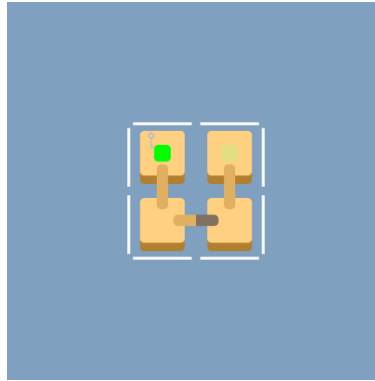


Figura 3: Nivel de tamaño 2×2 , el *jugador* ha tomado la *llave* de la *habitación* arriba-izquierda.
(Fuente: Elaboración propia)

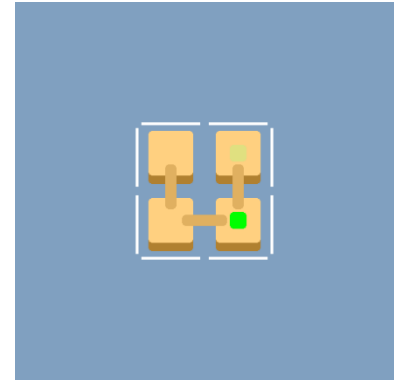


Figura 4: Nivel de tamaño 2×2 , el *jugador* atravesó la *conexión* bloqueada de la *habitación* abajo-derecha, consumiendo la *llave* que poseía en el proceso.
(Fuente: Elaboración propia)

Finalmente, la última regla del juego permite deslizar una *habitación* entre *casillas* si y solo si no existe un *jugador* sobre ella; la *casilla* a la que se deslizaría está vacía, es decir, no contiene otra *habitación*; y tampoco existe una *pared* entre las dos *casillas*:

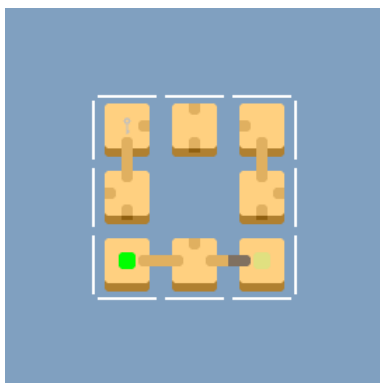


Figura 5: Nivel de tamaño 3×3 , cualquiera de las *habitaciones* cardinalmente adyacentes a la *casilla* vacía (en el centro) se pueden deslizar.
(Fuente: Elaboración propia)

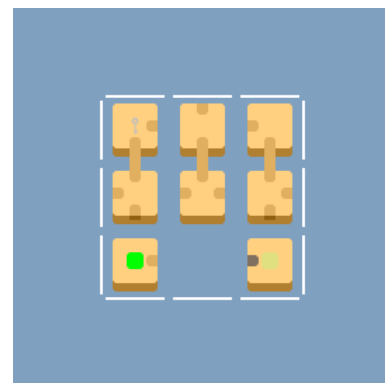


Figura 6: Nivel de tamaño 3×3 , solo las *habitaciones* arriba y a la derecha de la *casilla* vacía se pueden deslizar.
(Fuente: Elaboración propia)

Para efectos de este informe, se definieron características adicionales para los niveles estudiados, así, cada nivel:

- Siempre se puede completar
- Contiene exactamente un *jugador*
- Contiene exactamente una *salida*
- Contiene una cantidad equivalente de *llaves* y *conexiones* bloqueadas, garantizando siempre que tomar y utilizar una *llave* no impida la obtención y uso de al menos una siguiente si es que existe en el nivel
- Puede o no contener *casillas* que no se pueden utilizar dentro del rectángulo de tamaño $N \times M$

De esta manera, los niveles utilizados para estudiar el juego, además de los tres ya presentados, fueron los siguientes:

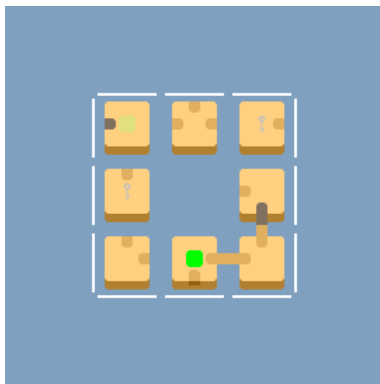


Figura 7: Nivel 4.

(Fuente: Elaboración propia)

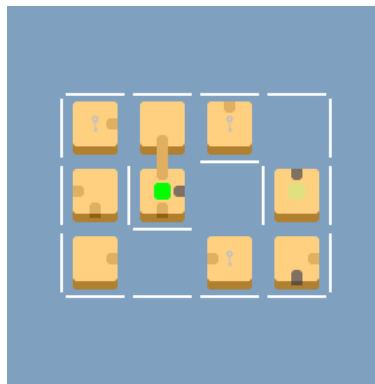


Figura 8: Nivel 5.

(Fuente: Elaboración propia)

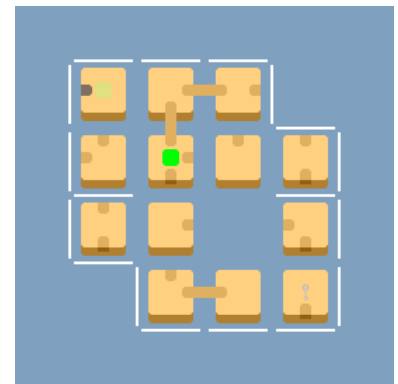


Figura 9: Nivel 6.

(Fuente: Elaboración propia)

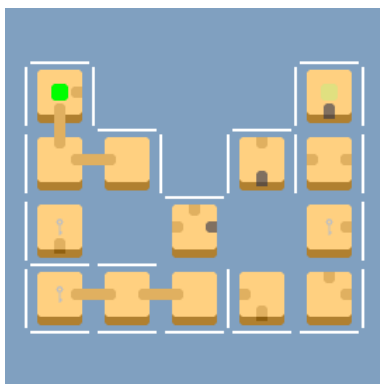


Figura 10: Nivel 7. (Fuente: Elaboración propia)

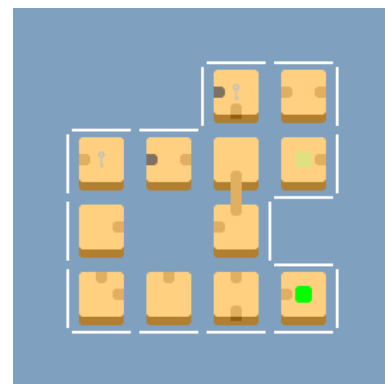


Figura 11: Nivel 8. (Fuente: Elaboración propia)

2. Motivación de la Investigación

Las motivaciones para haber realizado la presente investigación fueron varias: primero, se buscaba encontrar un algoritmo ya existente, o bien diseñar uno original, que, de manera fácilmente expandible (ya sea en tamaño, añadiendo reglas nuevas y/o ampliando las restricciones impuestas para esta investigación), pudiera determinar si un nivel dado es o no posible de completar, y si lo es, que retornara al menos una ruta óptima para hacerlo en menor tiempo que la fuerza bruta. Alternativamente, se buscaba comparar la eficiencia (definiendo esta para el caso del juego como la combinación entre la cantidad de acciones encontradas para completar un nivel, y el tiempo de ejecución del programa que se utilizara para encontrarlas) de algunos algoritmos conocidos para ayudar a dirigir la investigación hacia algoritmos y métodos que fueran más prometedores que los estudiados previamente, y así aumentar la posibilidad de encontrar un algoritmo o método que pueda cumplir con los requerimientos antes establecidos; o de diseñar uno original que sea mejor para este caso que las demás alternativas.

Segundo, con los resultados de esta investigación, se esperaba poder complementar el desarrollo de herramientas que analizaran en profundidad los niveles del juego, para así obtener información adicional sobre estos que ayudaran al diseño, desarrollo y control de calidad de niveles futuros, aportando por ejemplo: rutas óptimas, cuellos de botella de conjuntos de soluciones posibles, patrones y frecuencia de estados y tipos de soluciones, análisis y perfilado de jugadores reales en comparación con soluciones óptimas y otras conocidas, cambios mínimos requeridos para hacer posible un nivel, entre otras características.

Tercero, gracias al recorrido que implica la realización de la investigación, se buscaba obtener un mayor entendimiento de las propiedades y características del juego, lo que se puede entender como subrutinas óptimas generales para lograr cambios concretos dentro de un nivel: como intercambiar dos *habitaciones* sin alterar la configuración del resto, mover a un *jugador* con una cantidad limitada de *habitaciones* disponibles, expandir la cantidad de acciones posibles desde un estado dado, o rodear alguna cantidad y/o configuración de *conexiones* bloqueadas; también se pueden entender como patrones de acciones útiles para jugadores reales, reglas de decisión generales o específicas a ciertas configuraciones, estrategias generales para resolver niveles, métodos para comparar qué acciones son “mejores”, y más que faltaron por descubrir.

3. Marco Teórico

Para poder hacer la comparación entre algoritmos, primero se definió cuál sería la base mínima sobre la que comparar: la fuerza bruta, que actuó como *benchmark* inicial para comprender la escala del problema, y ser el mínimo que todos los otros algoritmos estudiados debieron superar.

Para realizar la fuerza bruta se eligieron dos métodos: *Depth First Search* (DFS) y *Breadth First Search* (BFS), ya que, tomando inspiración de un video de 2swap, en el que resuelve un juego diferente, se pudo comprender un nivel del juego como un grafo en el que cada vértice corresponde a un estado legal en este, y cada arista corresponde a la ejecución de alguna acción permitida dentro del nivel (2025) (ya sea mover a un *jugador* o deslizar una *habitación*), por lo que DFS y BFS fueron los mejores candidatos para comenzar el análisis del juego. DFS se eligió como método original para realizar el *benchmark* inicial de cada nivel, por la simpleza de la implementación recursiva (Cormen et al., 2022), y porque con la búsqueda en profundidad se podría explorar el fondo de los estados posibles de cada nivel, hasta el punto en el que sea imposible realizar una acción sin volver a un estado visitado previamente; además, se estimaba que en el caso de aumentar el tamaño de los niveles, la profundidad del grafo resultante crecería más rápido que el ancho del mismo. BFS se eligió como alternativa a DFS como *benchmark* inicial en el caso de que explorar el fondo de los estados posibles de cada nivel fuera menos temporalmente eficiente que explorar desde el inicio del nivel, además de que este método garantiza encontrar una, o todas, las soluciones óptimas de cada nivel (Cormen et al., 2022), por lo que el algoritmo ideal debe ser capaz de encontrar lo mismo que el BFS en al menos el mismo tiempo; además, este método puede encontrar el tamaño del grafo, es decir, la cantidad de estados únicos en el nivel, de una manera más ordenada que el DFS.

Luego de haber definido cómo se realizaría la fuerza bruta para obtener la *benchmark* inicial, se definió lo que podría ser prometedor tener en un algoritmo o método de tal manera que se pudiera priorizar su prueba frente a los niveles del juego por sobre otras opciones, así, un algoritmo o método ideal debe: generar comportamientos emergentes; evolucionar y/o encontrar las acciones que resulten más convenientes para completar un nivel, sin saber de antemano cuáles pueden ser esas acciones; generalizar lo aprendido (si es que aprende) a otros niveles y situaciones, o ser general para otros niveles y situaciones en primer lugar; ser fácilmente expandible para incorporar futuras nuevas reglas de juego; o garantizar

encontrar al menos una solución óptima, varias soluciones significativamente diferentes, o determinar rápidamente si un nivel es posible de completar o no.

Con lo anterior establecido, se optó por elegir el algoritmo genético como algoritmo a probar en profundidad para intentar resolver los niveles del juego, ya que cumple con la mayoría de las características deseables: puede generar comportamientos emergentes al evolucionar, lo cual puede significar que sea generalmente capaz de aprender cómo completar cada nivel, expandirlo requiere únicamente agregar las nuevas acciones que se vuelvan posibles y actualizar la función de evaluación, y puede encontrar múltiples soluciones que pueden ser muy diferentes entre sí. Además, el algoritmo genético tiene muchas partes que lo componen, específicamente: la creación de la generación inicial, la función de evaluación de los individuos, la selección de padres de una siguiente generación, el cruce de los padres para crear a una siguiente generación, y finalmente la mutación de los atributos de los individuos de la nueva generación (Gendreau & Potvin, 2010); la posibilidad de modificar de manera independiente cada una de estas cinco fases del algoritmo genético lo vuelve muy versátil y adaptable a cualquier circunstancia. Con el algoritmo elegido, se determinó que un individuo de una generación contendría únicamente una cadena de acciones: las que ese individuo realizó en el nivel, como los genes que se evaluarían y pasarían entre generaciones.

Lo siguiente que se determinó que debería hacerse con el algoritmo para probarlo en el contexto del juego fue: realizar un análisis de sensibilidad por método de factor único, utilizando como valores nominales de los parámetros aquellos que se descubrieron que generaron los mejores resultados (evaluados como la combinación de tiempo de ejecución, y la cantidad de acciones utilizadas para completar el nivel) después de realizar pruebas a mano, variando las fases más relevantes con métodos distintos, y los parámetros que cada combinación de métodos puede tener; porque así, se pudo determinar cuáles eran los parámetros más importantes para el éxito del algoritmo genético (Saltelli et al., 2007), y utilizar estos para la ejecución del algoritmo genético que se comparó con el *benchmark* inicial. Seguido a esto, se definieron las fases de cruce y mutación como las fases más relevantes a variar en el análisis de sensibilidad, debido al impacto directo que tienen ambas fases en modificar las acciones realizadas por los individuos entre generaciones, y, por ende, en la exploración que los individuos pueden realizar sobre el nivel que estén recorriendo. Se utilizaron los siguientes métodos para cada una de las dos fases:

En el caso de la fase de cruce, se determinó que los individuos de cada generación fueran emparejados y cruzados con algún otro con el que compartieran al menos un estado en el nivel, clonando la secuencia de acciones más pequeña de entre ambos padres hasta el primer estado compartido para ambos hijos, y manteniendo cada secuencia de acciones posterior al estado compartido utilizado de cada padre para cada hijo. Esto logra minimizar la cantidad de acciones requeridas para lograr el estado compartido, para así tener más de un solo individuo explorando desde un mismo estado más avanzado de un nivel:

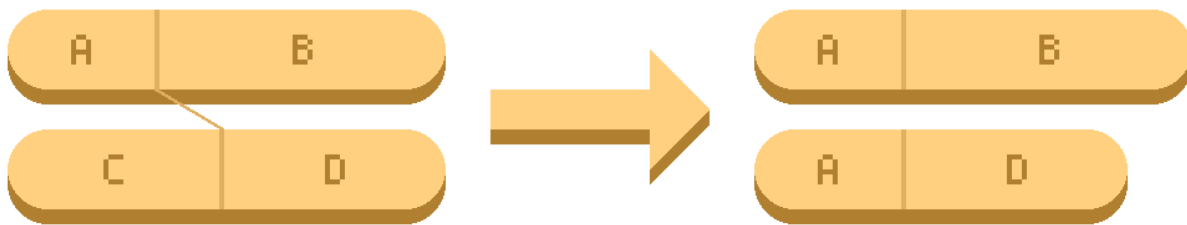


Figura 12: Representación visual de la fase de cruce con dos individuos que comparten un estado (línea que divide cada individuo en dos grupos de acciones), donde el primer individuo se clona para la siguiente generación, pero el segundo reemplaza su grupo de acciones C por el grupo de acciones A del primer individuo. (Fuente: Elaboración propia)

Para aquellos individuos que no encontrasen estados compartidos con ningún otro individuo sin emparejar, se definieron cuatro métodos de respaldo diferentes, por la variedad de resultados y comportamientos que se infirieron podrían emerger a partir de estos:

- 1) **BFS Bidireccional:** Se eligen pares de individuos al azar, y para cada par de individuos, se elige un estado aleatorio dentro de los visitados de cada uno de los dos, y se ejecuta el algoritmo BFS Bidireccional para encontrar las acciones necesarias para unir ambos estados elegidos, de esta manera, un hijo tendrá la secuencia inicial de acciones que llegan al estado elegido del primer padre, la secuencia de acciones encontrada para unirlo con el estado elegido del segundo padre, y la secuencia de acciones que sigan al estado elegido del segundo padre; mientras que el segundo hijo será un clon del padre mejor evaluado por la función de evaluación:

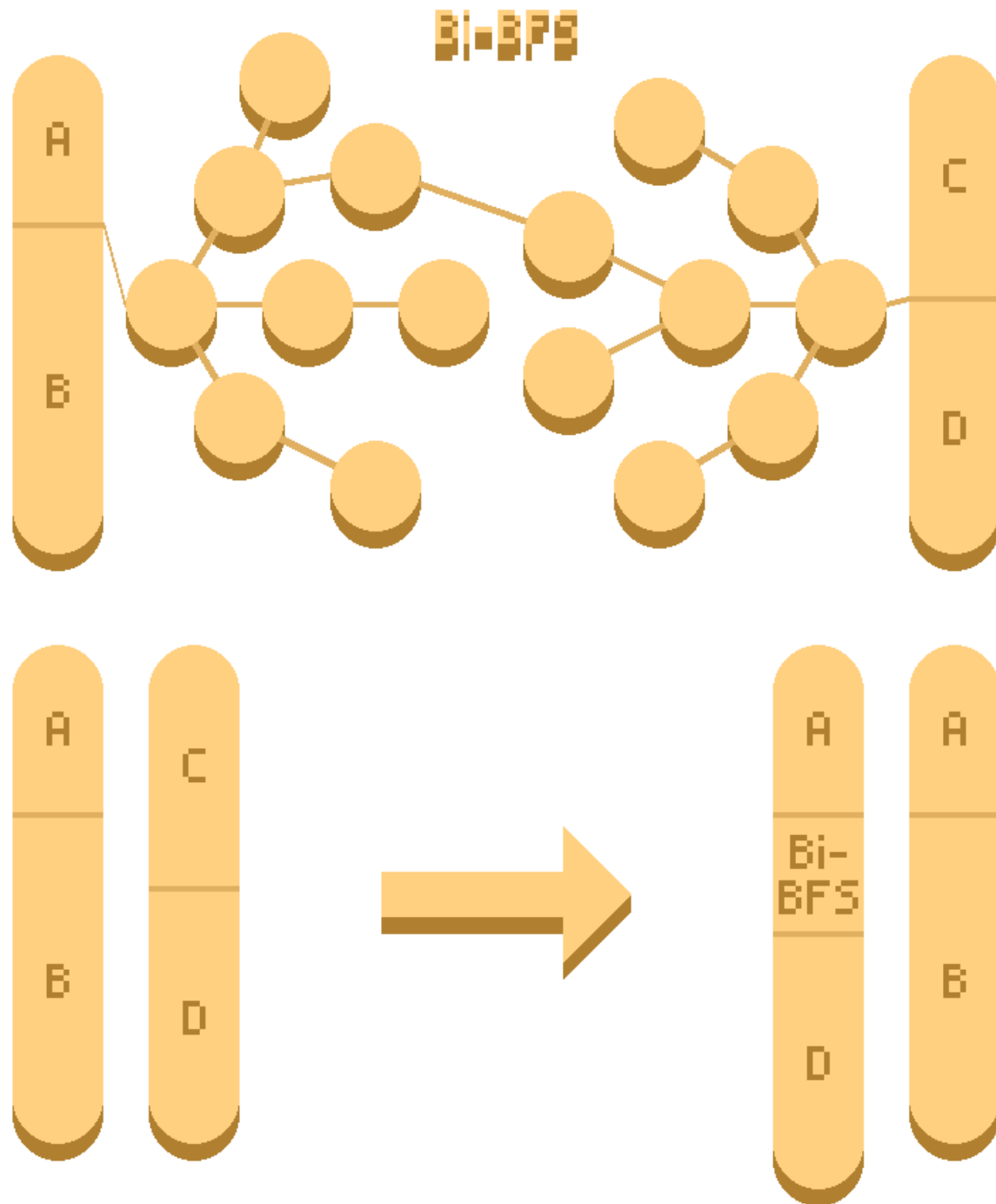


Figura 13: Representación visual del BFS Bidireccional, uniendo los estados aleatoriamente elegidos de ambos individuos, construyendo así al nuevo primer individuo (izquierda de los resultantes). El segundo nuevo individuo corresponde al clon del individuo mejor evaluado por la función de evaluación.

(Fuente: Elaboración propia)

- 2) Corte y Generación: Se corta la secuencia de acciones de cada individuo en algún punto aleatorio para cada uno, o se eliminan las últimas N acciones realizadas por cada individuo, y se generan acciones válidas aleatorias desde el estado final posterior al corte hasta recuperar el largo original de la secuencia de acciones de cada individuo:

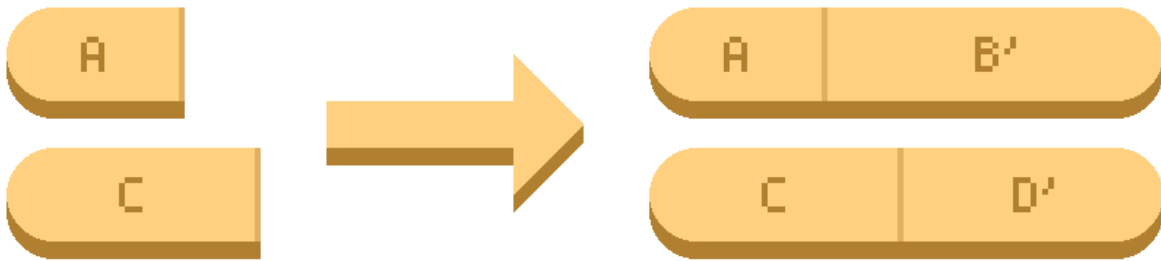


Figura 14: Representación visual de la versión del método Corte y Generación en la que se corta la secuencia de acciones en un punto aleatorio a cada individuo del par elegido para luego generar acciones válidas aleatorias hasta recuperar el largo original que cada uno tenía. (Fuente: Elaboración propia)

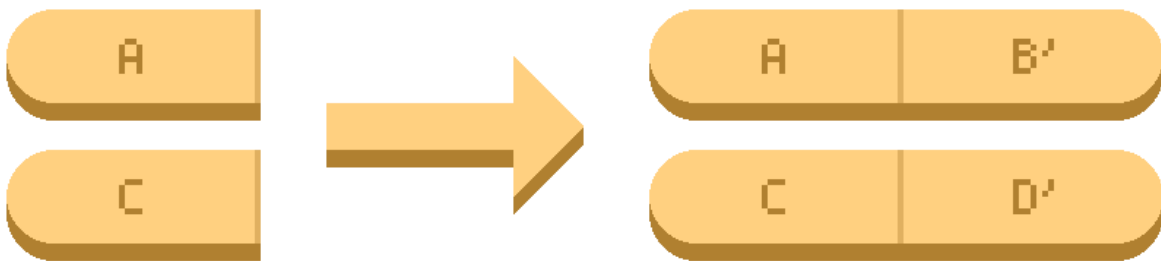


Figura 15: Representación visual de la versión del método Corte y Generación en la que se eliminan las últimos N acciones realizadas por cada individuo, y se generan acciones válidas aleatorias hasta recuperar el largo original que cada uno tenía. (Fuente: Elaboración propia)

- 3) Regeneración: Cada individuo es reemplazado por un nuevo individuo generado aleatoriamente con la misma cantidad de acciones realizadas que el individuo anterior:

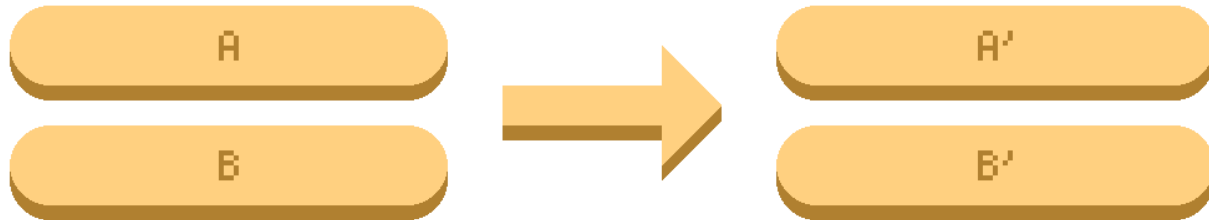


Figura 16: Representación visual del método Regeneración, crea nuevos individuos con la misma cantidad de acciones que los originales para reemplazarlos en la nueva generación. (Fuente: Elaboración propia)

- 4) Clonación: Se mantienen todos los individuos para la siguiente generación:

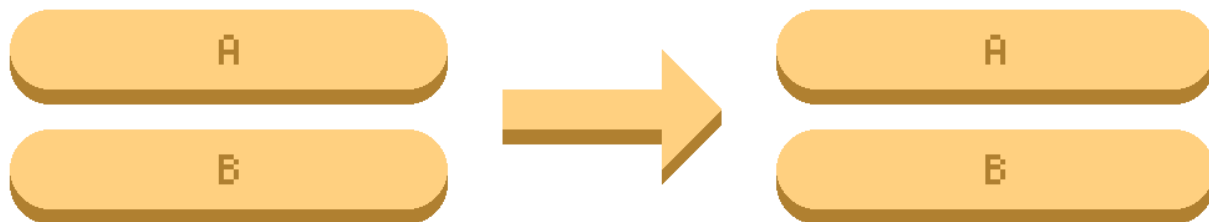


Figura 17: Representación visual del método Clonación, los individuos se mantienen para la siguiente generación. (Fuente: Elaboración propia)

En el caso de la fase de mutación, se definieron dos métodos:

- 1) Final Aleatorio: Cada individuo realiza N acciones válidas aleatorias adicionales desde el estado final que alcanzó:

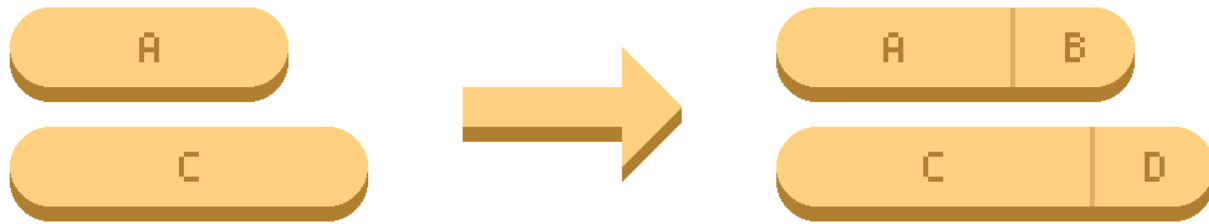


Figura 18: Representación visual del método Final Aleatorio, tanto el primer como el segundo individuo generaron la misma cantidad de acciones válidas aleatorias (no necesariamente las mismas entre ellos).

(Fuente: Elaboración propia)

- 2) Corte y Generación: Se corta la secuencia de acciones de cada individuo en algún punto aleatorio para cada uno, o se eliminan las últimas N acciones realizadas por cada individuo, y se generan acciones válidas aleatorias desde el estado final posterior al corte hasta recuperar el largo original de la secuencia de acciones de cada individuo. (Mismo método que el segundo método de respaldo de la fase de cruce.)

4. Resultados

En forma de resultados generales, se obtuvieron de los *benchmarks* realizados con DFS los siguientes valores:

DFS	Nivel 1	Nivel 2	Nivel 3	Nivel 4
Cantidad de acciones utilizadas	2	4	8	24
Tiempo de ejecución del algoritmo (en segundos)	0,001	0,001	0,129	14.513,806

Tabla 1: Resultados del *benchmark* realizado con DFS. El algoritmo solo logró completar los primeros cuatro niveles. (Fuente: Elaboración propia)

DFS	Nivel 5	Nivel 6	Nivel 7	Nivel 8
Profundidad máxima explorada	14	19	12	24
Tiempo de ejecución del algoritmo (en segundos)	361.389,376	109.058,678	37.618,935	38.132,594

Tabla 2: Resultados del *benchmark* realizado con DFS. Se muestran las máximas profundidades y el tiempo que el algoritmo utilizó en esos niveles antes de detener el *benchmark*. (Fuente: Elaboración propia)

Como parte del estudio, se eligieron los niveles 3 y 4 para que el algoritmo DFS buscara una solución posible en cada capa de profundidad de la búsqueda a partir de la primera en encontrar una solución posible (ya que estos corresponden al final del tutorial y al primer nivel normal, respectivamente), que resultaron ser las profundidades 8 y 24 para cada uno de los dos niveles, respectivamente.

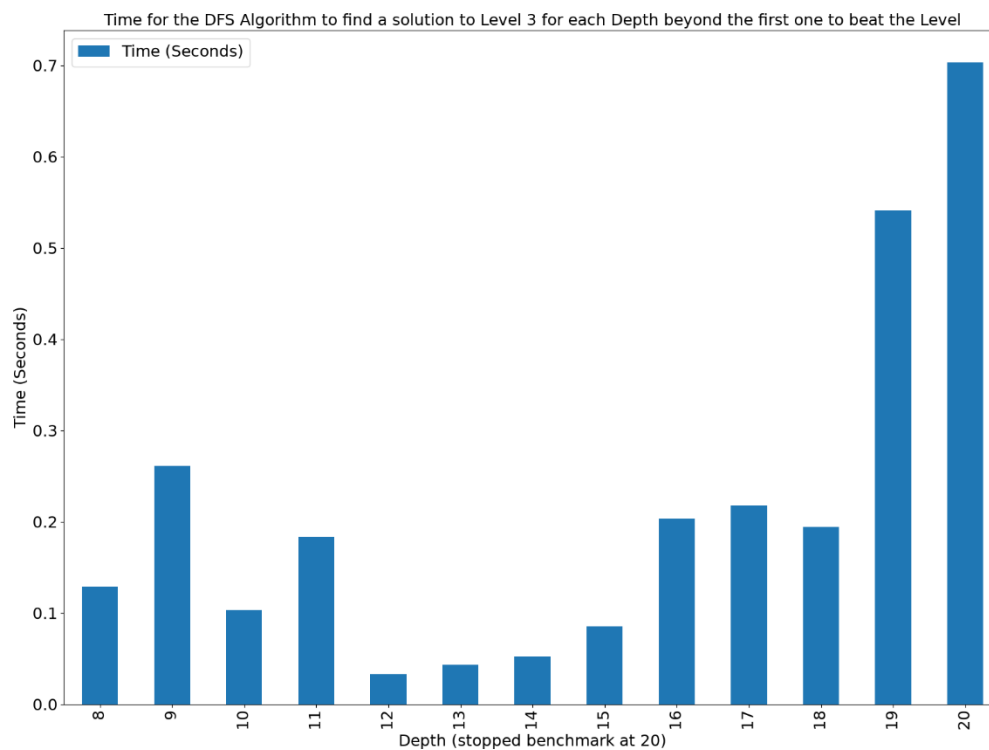


Gráfico 1: Tiempo requerido por el algoritmo DFS en cada profundidad explorada antes de detener el *benchmark* desde la primera para encontrar una solución al nivel 3. (Fuente: Elaboración propia)

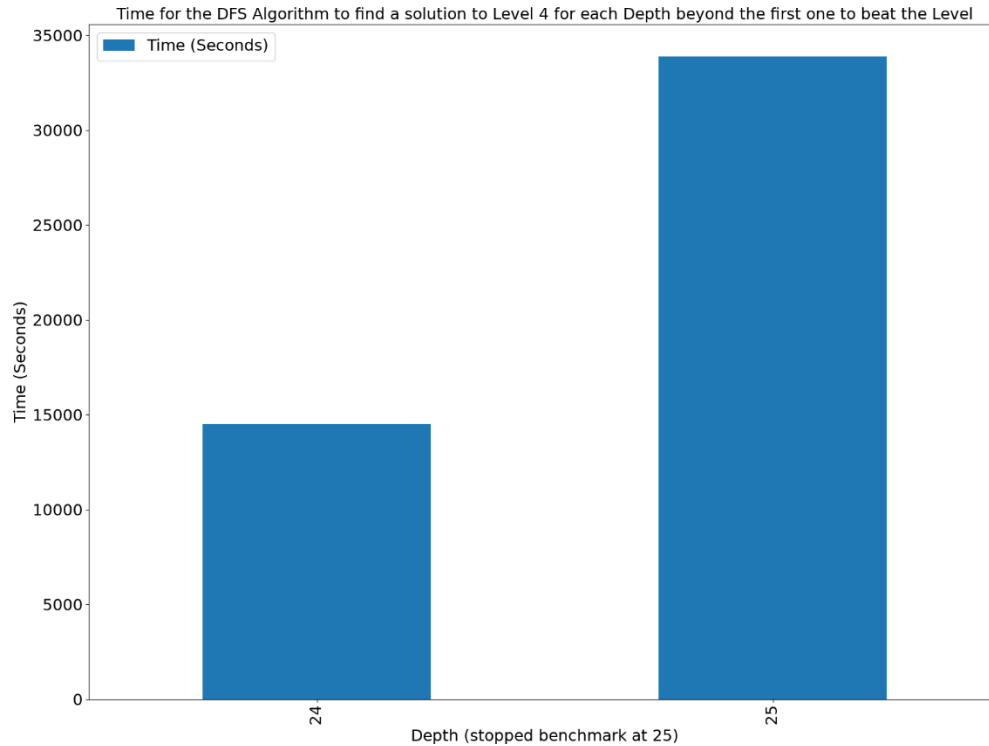


Gráfico 2: Tiempo requerido por el algoritmo DFS en cada profundidad explorada antes de detener el *benchmark* desde la primera para encontrar una solución al nivel 4. (Fuente: Elaboración propia)

Cabe destacar que el algoritmo DFS superó el límite estándar de recursión de 1.000 llamadas en Python, lenguaje usado para escribir la simulación del juego (A.2), por lo que se tuvo que aumentar a 1.000.000 para que el algoritmo lograra completar los últimos niveles que sí logró resolver.

En forma de resultados generales, se obtuvieron de los *benchmarks* realizados con BFS los siguientes valores:

BFS	Nivel 1	Nivel 2	Nivel 3	Nivel 4	Nivel 5	Nivel 6	Nivel 7
Cantidad de acciones necesarias para una solución óptima	3	4	8	24	47	30	45

Tiempo de ejecución del algoritmo (en segundos)	0,001	0,001	0,039	39,985	4.281,119	20.542,598	100.191,799
---	-------	-------	-------	--------	-----------	------------	-------------

Tabla 3: Resultados del *benchmark* realizado con BFS. El algoritmo logró completar todos los niveles, a excepción del último. (Fuente: Elaboración propia)

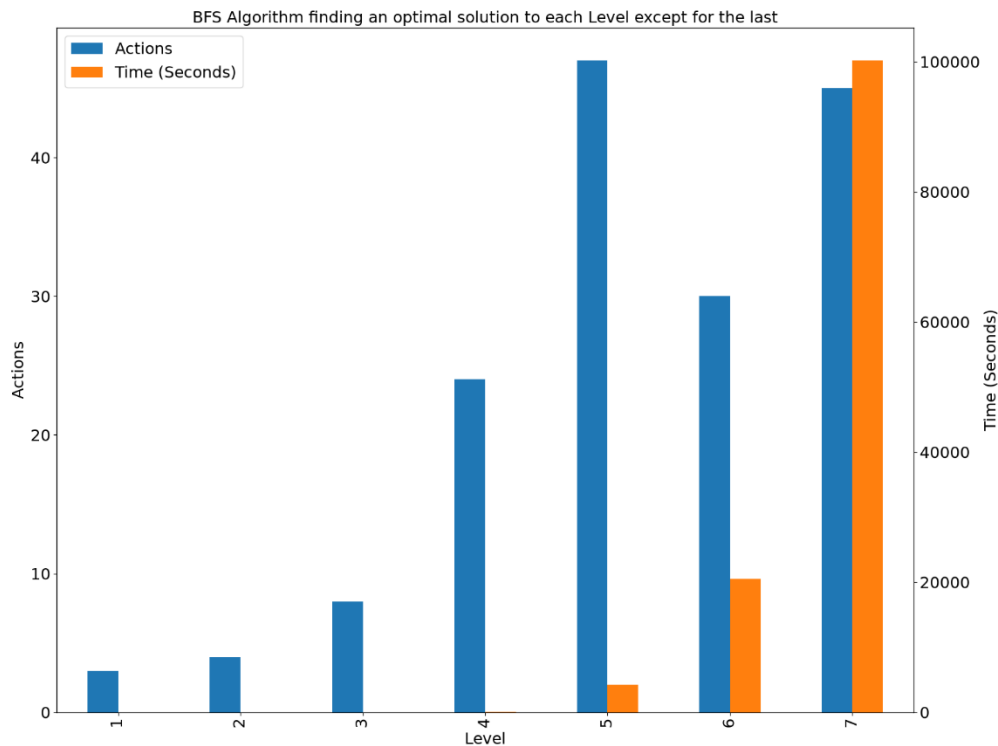


Gráfico 3: Visualización de la Tabla 3. (Fuente: Elaboración propia)

Profundidad	Cantidad de estados únicos	Tiempo (en segundos)
1	2	0,001
2	3	0,001

Tabla 4: Tamaño del espacio de búsqueda del nivel 1 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 1, el tamaño completo del espacio de búsqueda son 3 estados únicos. (Fuente: Elaboración propia)

Profundidad	Cantidad de estados únicos	Tiempo (en segundos)
1	2	0,001
2	3	0,001
3	4	0,001
4	6	0,001
5	7	0,001

Tabla 5: Tamaño del espacio de búsqueda del nivel 2 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 2, el tamaño completo del espacio de búsqueda son 7 estados únicos. (Fuente: Elaboración propia)

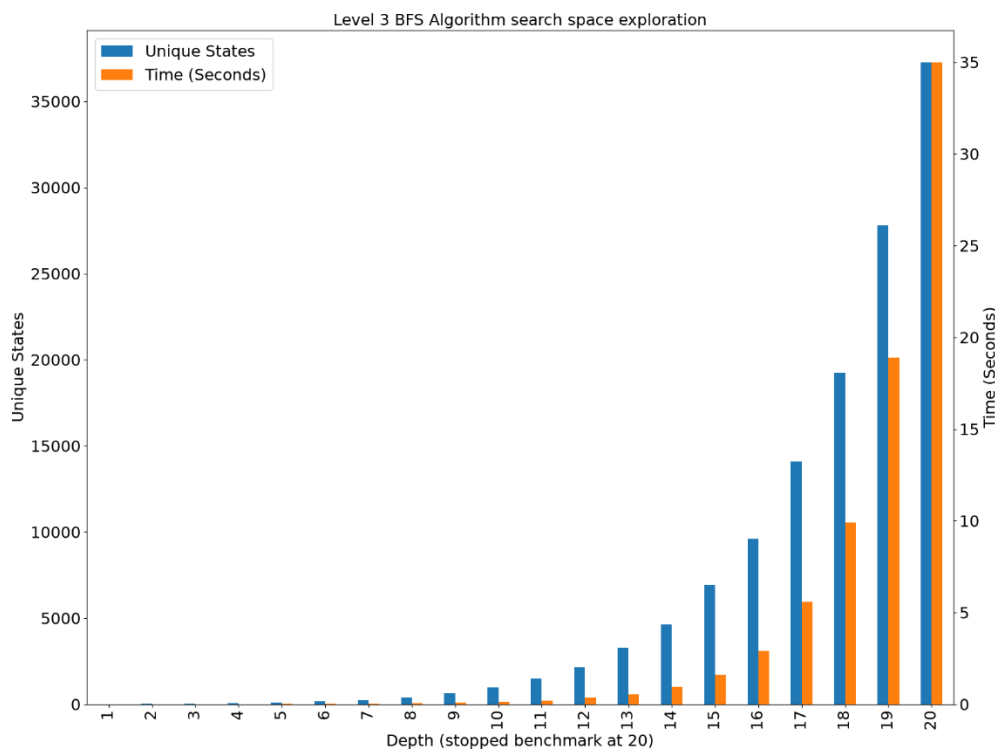


Gráfico 4: Tamaño del espacio de búsqueda del nivel 3 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 3, el algoritmo no encontró el tamaño completo del espacio de búsqueda antes de detener el *benchmark*. (Fuente: Elaboración propia)

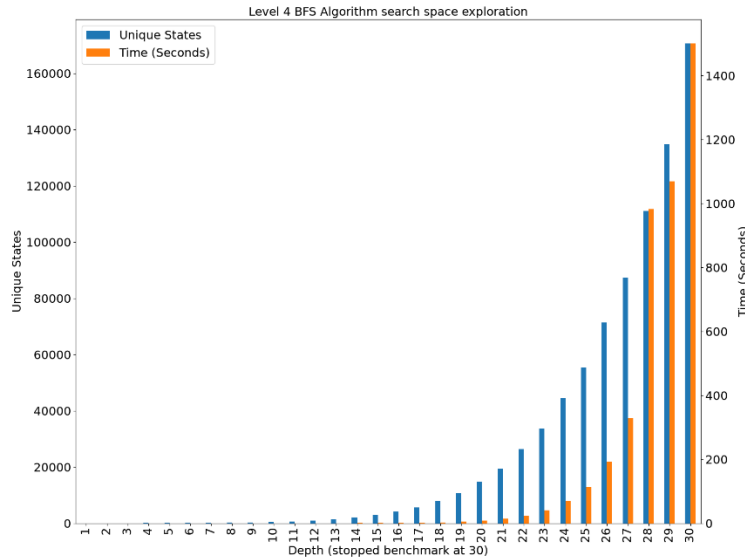


Gráfico 5: Tamaño del espacio de búsqueda del nivel 4 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 4, el algoritmo no encontró el tamaño completo del espacio de búsqueda antes de detener el *benchmark*. (Fuente: Elaboración propia)

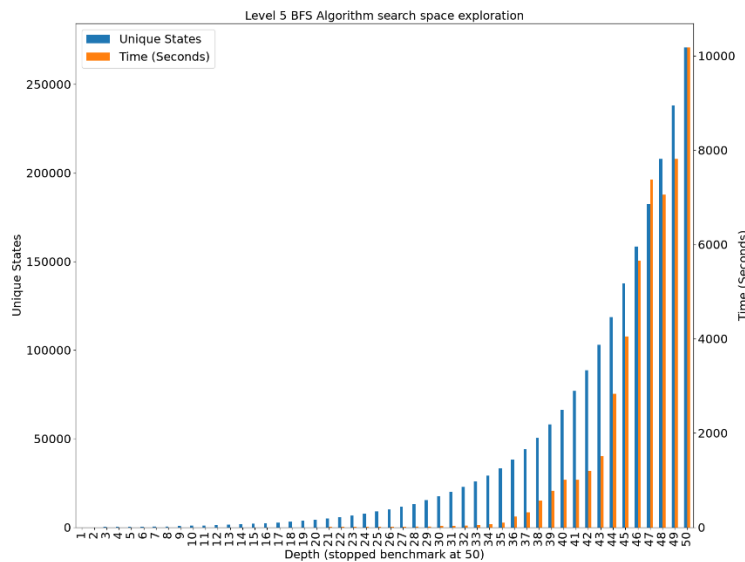


Gráfico 6: Tamaño del espacio de búsqueda del nivel 5 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 5, el algoritmo no encontró el tamaño completo del espacio de búsqueda antes de detener el *benchmark*. (Fuente: Elaboración propia)

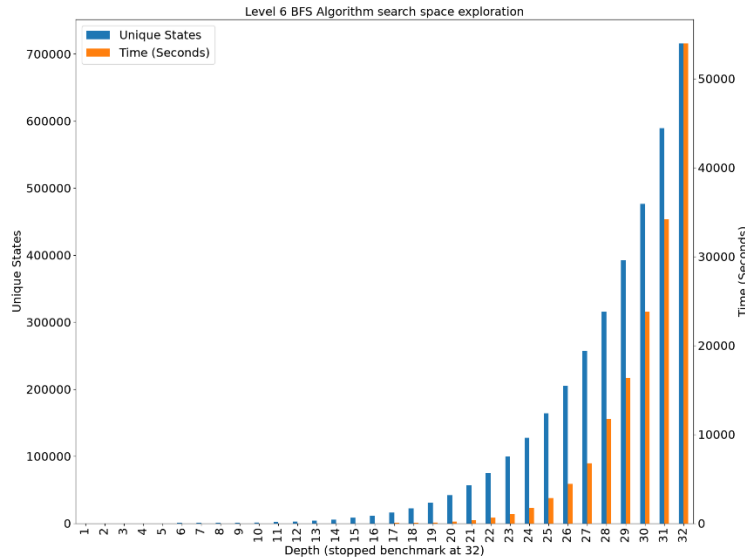


Gráfico 7: Tamaño del espacio de búsqueda del nivel 6 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 6, el algoritmo no encontró el tamaño completo del espacio de búsqueda antes de detener el *benchmark*. (Fuente: Elaboración propia)

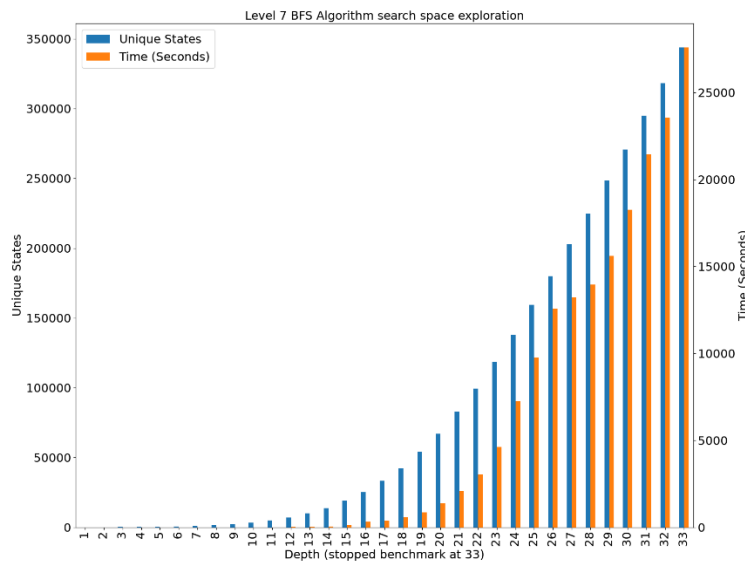


Gráfico 8: Tamaño del espacio de búsqueda del nivel 7 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 7, el algoritmo no encontró el tamaño completo del espacio de búsqueda antes de detener el *benchmark*. (Fuente: Elaboración propia)

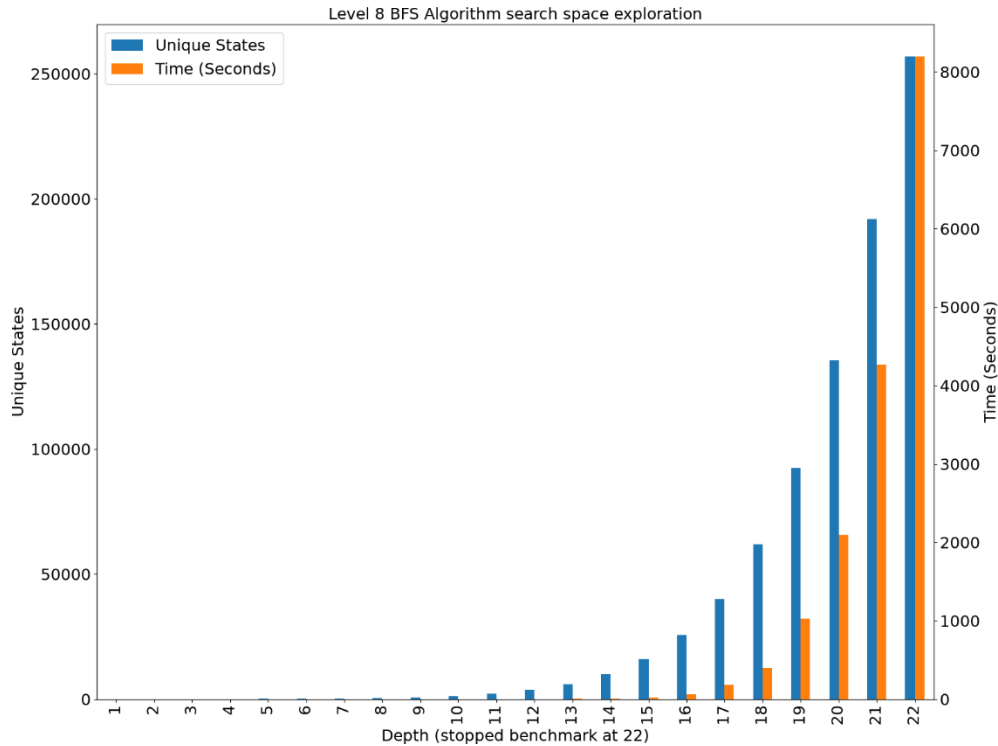


Gráfico 9: Tamaño del espacio de búsqueda del nivel 8 obtenido con el algoritmo BFS. Muestra la cantidad de estados únicos que existen en el nivel hasta cada profundidad máxima de búsqueda. Para el nivel 8, el algoritmo no encontró el tamaño completo del espacio de búsqueda antes de detener el *benchmark*. (Fuente: Elaboración propia)

4.1 Implementación del Algoritmo Genético

La implementación del algoritmo genético implicó la definición de sus cinco fases: la creación de la generación inicial, la evaluación de los individuos, la selección de padres de una siguiente generación, el cruce de los padres para crear a una siguiente generación, y la mutación de los atributos de los individuos de la nueva generación, cada una se describe a continuación, junto con todos los métodos y parámetros que cada uno puede tener, y pseudocódigo de su implementación en la simulación del juego:

- Creación de la generación inicial: Se implementó un único método, el cual genera individuos tomando acciones aleatoriamente. De esta fase se extraen dos parámetros:
 - *Generation_Size*: La cantidad de individuos de la generación.
 - *Actions_Amount*: La cantidad de acciones que se generarán para cada individuo.

```
function Create_Generation(_Parameters, _Level_ID) {
    set Generation to empty list

    for (set _ to 0; _ < _Parameters.Generation_Size; _++) {
        set Individual_Game to new Game(_Level_ID)
        set _Individual to new Individual()

        for (set __ to 0; __ < _Parameters.Actions_Amount; __++) {
            set Chosen_Action to choose with equal Weights from
            Individual_Game.Decision_Space

            execute Chosen_Action in Individual_Game
            append Chosen_Action to _Individual.Action_History
        }

        append _Individual to Generation
    }

    return Generation
}
```

- Evaluación de los individuos: Se implementó un único método, el cual calcula la evaluación del individuo que evalúe según las características de la partida y de eventos que hayan ocurrido durante esta. De esta fase se extraen cinco parámetros:
 - *Length_Weight*: Valor de una acción realizada en el nivel, se multiplica por la cantidad de acciones que realice el individuo.
 - *Obtained_Keys_Bonus*: Bonificación que se suma cuando el individuo obtiene una llave en el nivel.
 - *Used_Keys_Bonus*: Bonificación que se suma cuando el individuo utiliza una llave en el nivel para desbloquear una conexión.
 - *Has_Won_Bonus*: Bonificación que se suma cuando el individuo completa el nivel (llevando al jugador a la casilla con la salida)
 - *Has_Looped_Penalty*: Penalización que se suma cuando el individuo ha alcanzado un ciclo dentro del nivel (es decir, ha vuelto a un estado que ya ha visitado antes).

```
function Fitness_Gameplay(_Individual, _Parameters, _Level_ID) {
  set Individual_Game to new Game(_Level)
  set Visited_States to list with: state hash of Individual_Game
  set Fitness_Score to 0

  for (set _Action_Index to 0; _Action_Index < length of
    _Individual.Action_History; _Action_Index++) {
    set _Action to _Individual.Action_History[_Action_Index]

    if (_Action.Type is CONSTANT.Explorer_Move) {
      set Required_Keys to Required Keys of Individual_Game doing _Action

      Fitness_Score += _Parameters.Used_Keys_Bonus * Required_Keys

      if (_Action obtained Key) {
        Fitness_Score += _Parameters.Obtained_Keys_Bonus
      }
    }

    execute _Action in Individual_Game

    set Current_State to state hash of Individual_Game

    if (Current_State in Visited_States) {
      Fitness_Score += _Parameters.Has_Looped_Penalty
      break
    }

    if (Individual_Game has Won) {
      Fitness_Score += _Parameters.Has_Won_Bonus
      break
    }

    append Current_State to Visited_States
  }

  Fitness_Score += _Parameters.Length_Weight * (_Action_Index + 1)

  return Fitness_Score
}
```

- Selección de padres de una siguiente generación: Se implementó un único método, el cual utiliza el modelo de suma ponderada para asignar las probabilidades de selección de cada individuo (Bozorg-Haddad et al., 2021), con la precaución de evitar pesos negativos, incluso si todos los individuos en la generación obtuvieron una evaluación negativa (porque todos encontraron ciclos). De esta fase no se extrae ningún parámetro.

```
function Select_Parents(_Generation) {  
  if (all _Individual.Fitness_Score in _Generation <= 0) {  
    set Worst_Fitness to minimum of _Individual.Fitness.Score in _Generation  
  
    if (all _Individual.Fitness_Score - Worst_Fitness in _Generation == 0) {  
      return choose with equal Weights from _Generation length of _Generation  
times with replacement  
    }  
  
    return choose with Weights equal to each _Individual.Fitness_Score -  
Worst_Fitness of _Generation from _Generation length of _Generation times with  
replacement  
  }  
  
  return choose with Weights equal to each maximum between  
_Individual.Fitness_Score and 0 of _Generation from _Generation length of  
_Generation times with replacement  
}
```

- Cruce de padres: Se implementó un único método, el cual busca formar parejas de individuos que compartan a lo menos un estado del nivel, para así utilizar ese estado como *pivot* para alterar las secuencias de acciones de uno de los padres, como se mostró en la Figura 12. De esta fase se extraen dos parámetros:
 - *Search_Start_Percent*: Porcentaje de la secuencia de acciones de cada individuo desde el que puede comenzar la búsqueda por un estado común.
 - *Search_End_Percent*: Porcentaje de la secuencia de acciones de cada individuo hasta el que puede alcanzar la búsqueda por un estado común.

```

function Crossover Fusion(_Generation, _Parameters, _Level_ID) {
    set Next_Generation to empty list
    set Processed_Indices to empty list

    for (set i_A to 0; i_A < length of _Generation; i_A++) {
        for (set i_B to i_A + 1; i_B < length of _Generation; i_B++) {
            if (_Generation[i_A] is same as _Generation[i_B] or i_A is in
Processed_Indices or i_B is in Processed_Indices) {
                continue
            }

            set Game_A to new Game(_Level_ID)
            set Game_B to new Game(_Level_ID)
            set Start_A_Index to integer(_Parameters.Search_Start_Percent * (length
of _Generation[i_A].Action_History - 1))
            set Start_B_Index to integer(_Parameters.Search_Start_Percent * (length
of _Generation[i_B].Action_History - 1))

            for (set i to 0; i < Start_A_Index) {
                execute _Generation[i_A].Action_History[i] in Game_A
            }

            for (set j to 0; j < Start_B_Index; j++) {
                execute _Generation[i_B].Action_History[j] in Game_B
            }

            set Saved_States_A to empty list
            set Saved_States_B to empty list

            for (set k to Start_A_Index; k < integer(_Parameters.Search_End_Percent
* length of _Generation[i_A].Action_History); k++) {
                execute _Generation[i_A].Action_History[k] in Game_A
                append state hash of Game_A to Saved_States_A
            }

            for (set l to Start_B_Index; l < integer(_Parameters.Search_End_Percent
* length of _Generation[i_B].Action_History); l++) {
                execute _Generation[i_B].Action_History[l] in Game_B
                append state hash of Game_B to Saved_States_B
            }
        }
    }
}

```

```

        for (set State_A_Index to 0; State_A_Index < length of Saved_States_A;
State_A_Index++) {
            if (Saved_States_A[State_A_Index] is in Saved_States_B) {
                set State_B_Index to Saved_States_A[State_A_Index] index inside
Saved_States_B
                set Is_Cloned_Individual_A to boolean result of Start_A__Index +
State_A_Index <= Start_B__Index + State_B_Index
                set Cloned_Individual to new Individual(value copy of
_Generation[i_A].Action_History if Is_Cloned_Individual_A else value copy of
_Generation[i_B].Action_History)

                append Cloned_Individual to Next_Generation

                if (Is_Cloned_Individual_A) {
                    append new Individual(Cloned_Individual.Action_History[until
excluding Start_A__Index + State_A_Index + 1] joined with
_Generation[i_B].Action_History[starting from Start_B__Index + State_B_Index + 1])
to Next_Generation
                }
                else {
                    append new Individual(Cloned_Individual.Action_History[until
excluding Start_B__Index + State_B_Index + 1] joined with
_Generation[i_A].Action_History[starting from Start_A__Index + State_A_Index + 1])
to Next_Generation
                }

                append i_A to Processed_Indices
                append i_B to Processed_Indices
                break
            }
        }

        if (length of Processed_Indices == length of _Generation) {
            return Next_Generation
        }

        execute _Parameters.Fallback_Method
    }

```

Sin embargo, este método necesita un método de respaldo para los individuos que no compartan ningún estado con los individuos restantes, este método de respaldo se extrae como un parámetro, junto con los parámetros que este método pueda tener, por lo que en total este método tiene tres parámetros: *Search_Start_Percent*, *Search_End_Percent*, y *Fallback_Method* (este último es un *struct* con el método de respaldo y los parámetros correspondientes para este). Se implementaron cuatro métodos de respaldo:

- BFS Bidireccional: Elige un estado aleatorio para cada Individuo en cada par, y ejecuta el algoritmo BFS desde ambos estados hasta encontrar un estado común, para construir a un nuevo individuo con la ruta intermedia encontrada, como se mostró en la Figura 13, y clona al individuo con mejor evaluación del par. A continuación se presenta el pseudocódigo del algoritmo BFS Bidireccional implementado para la simulación del juego, previo al pseudocódigo del método de respaldo que lo utiliza:

```
function Bi_BFS(_Game_A, _Game_B, _Max_Depth) {
  set Found_Solution to boolean result of state hash of _Game_A == state hash of
_Game_B
  set Initialized_First_Queues to false
  set Current_Depth_A to 1
  set Current_Depth_B to 1
  set Connection_Index_A to -1
  set Connection_Index_B to -1
  set Action_History_A to empty list
  set Action_History_B to empty list
  set Queue_A to empty list
  set Queue_B to empty list
  set Visited_States_A to empty list
  set Visited_States_B to empty list

  if (Found_Solution) {
    return empty list
  }

  while (not Found_Solution) {
    if (_Max_Depth != -1 and Current_Depth_A > _Max_Depth) {
      return undefined
    }
  }
```

```
if (not Initialized_First_Queues) {
    set Initialized_First_Queues to true

    append new State(state hash of _Game_A, value copy of Action_History_A)
to Visited_States_A
    append new State(state hash of _Game_B, value copy of Action_History_B)
to Visited_States_B

    foreach (_Action in _Game_A.Decision_Space) {
        append _Action to Action_History_A
        append value copy of Action_History_A to Queue_A
        pop last of Action_History_A
    }

    foreach (_Action in _Game_B.Decision_Space) {
        append _Action to Action_History_B
        append value copy of Action_History_B to Queue_B
        pop last of Action_History_B
    }
}

while (length of Queue_A > 0 and length of Queue_A[0] == Current_Depth_A) {
    set Target_History_A to pop first of Queue_A

    while (length of Action_History_A > 0 and Action_History_A !=
Target_History_A[until excluding length of Action_History_A]) {
        undo pop last of Action_History_A in _Game_A
    }

    foreach (_Action in Target_History_A[starting from length of
Action_History_A]) {
        execute _Action in _Game_A
    }

    set Current_State_A to state hash of _Game_A
    set Action_History_A to Target_History_A

    if (any _State in Visited_States_A == Current_State_A) {
        continue
    }

    append new State(Current_State_A, value copy of Action_History_A) to
Visited_States_A
```

```

    foreach (_State in Visited_States_B) {
        if (Current_State_A == _State) {
            set Found_Solution to true
            set Connection_Index_B to _State index inside Visited_States_B
            break
        }
    }

    if (Found_Solution) {
        break
    }

    foreach (_Action in _Game_A.Decision_Space) {
        append _Action to Action_History_A
        append value copy of Action_History_A to Queue_A
        pop last of Action_History_A
    }
}

if (Found_Solution) {
    break
}

while (length of Queue_B > 0 and length of Queue_B[0] == Current_Depth_B) {
    set Target_History_B to pop first of Queue_B

    while (length of Action_History_B > 0 and Action_History_B !=
Target_History_B[until excluding length of Action_History_B]) {
        undo pop last of Action_History_B in _Game_B
    }

    foreach (_Action in Target_History_B[starting from length of
Action_History_B]) {
        execute _Action in _Game_B
    }

    set Current_State_B to state hash of _Game_B
    set Action_History_B to Target_History_B

    if (any _State in Visited_States_B == Current_State_B) {
        continue
    }

    append new State(Current_State_B, value copy of Action_History_B) to
Visited_States_B

```

```
    foreach (_State in Visited_States_A) {
        if (Current_State_B == _State) {
            set Found_Solution to true
            set Connection_Index_A to _State index inside Visited_States_A
            break
        }
    }

    if (Found_Solution) {
        break
    }

    foreach (_Action in _Game_B.Decision_Space) {
        append _Action to Action_History_B
        append value copy of Action_History_B to Queue_B
        pop last of Action_History_B
    }
}

Current_Depth_A += 1
Current_Depth_B += 1
}

return Visited_States_A[Connection_Index_A].Action_History joined with reversed
_Action foreach _Action in reversed
Visited_States_B[Connection_Index_B].Action_History
}
```

```

function Fallback_Bi_BFS(_Generation, _Next_Generation, _Processed_Indices,
_Parameters, _Level_ID) {
    for (set i_A to 0; i_A < length of _Generation; i_A++) {
        for (set i_B to i_A + 1; i_B length of _Generation; i_B++) {
            if (i_A is in _Processed_Indices or i_B is in _Processed_Indices) {
                continue
            }

            set Game_A to new Game(_Level_ID)
            set Game_B to new Game(_Level_ID)
            set State_A_Index to random integer between 1 and length of
            _Generation[i_A].Action_History
            set State_B_Index to random integer between 0 and length of
            _Generation[i_B].Action_History - 1

            for (set i to 0; i < State_A_Index; i++) {
                execute _Generation[i_A].Action_History[i] in Game_A
            }
            for (set j to 0; j < State_B_Index; j++) {
                execute _Generation[i_B].Action_History[j] in Game_B
            }

            append new Individual(value copy of _Generation[i_A].Action_History if
            _Generation[i_A].Fitness_Score >= _Generation[i_B].Fitness_Score else value copy of
            _Generation[i_B].Action_History) to _Next_Generation

            set Bi_BFS_Actions to Bi_BFS(Game_A, Game_B,
            _Parameters.Fallback_Parameters.Max_Depth)

            if (Bi_BFS_Actions) {
                append new Individual(_Generation[i_A].Action_History[until
excluding State_A_Index] joined with Bi_BFS_Actions joined with
            _Generation[i_B].Action_History[starting from State_B_Index]) to _Next_Generation
            }
            else {
                append new Individual(value copy of _Generation[i_A].Action_History
if _Generation[i_A].Fitness_Score < _Generation[i_B].Fitness_Score else value copy
of _Generation[i_B].Action_History) to _Next_Generation
            }

            append i_A to _Processed_Indices
            append i_B to _Processed_Indices
        }
    }
}

```

De este método de respaldo solo se extrae un parámetro:

- *Max_Depth*: Profundidad máxima permitida para que el algoritmo busque un estado intermedio. Si se supera este límite, el método resultará en la clonación de ambos individuos en el par.
- Corte y Generación: Ejecuta el método de mutación Corte y Generación, que dependiendo de su modo, corta la secuencia de acciones de cada individuo en algún punto aleatorio diferente para cada uno, y genera acciones aleatorias válidas hasta que cada individuo recupere su cantidad original de acciones; o elimina las últimas N acciones de cada individuo, y genera acciones aleatorias válidas hasta que cada individuo recupere su cantidad original de acciones, como se mostró en la Figura 14 y 15. A continuación se presenta el pseudocódigo del método de mutación Corte y Generación implementado para la simulación del juego, previo al pseudocódigo del método de respaldo que lo utiliza:

```

function Mutate_Cut_And_Generation(_Individual, _Parameters, _Level_ID) {
  set Individual_Game to new Game(_Level_ID)
  set Original_Action_Length to length of _Individual.Action_History

  if (_Individual has looped) {
    set _Individual.Action_History to empty list
  }
  else {
    if (_Parameters.Random_Cut_Start_Percent != -1) {
      set _Individual.Action_History to _Individual.Action_History[until
excluding random integer between integer(_Parameters.Random_Cut_Start_Percent *
length of _Individual.Action_History) and length of _Individual.Action_History]
    }
    else {
      set _Individual.Action_History to _Individual.Action_History[until
excluding maximum between 0 and length of _Individual.Action_History -
_Parameters.Last_Actions_To_Change]
    }
  }

  for (set _Action_Index to 0; _Action_Index < length of
_Individual.Action_History; _Action_Index++) {
    execute _Individual.Action_History[_Action_Index] in Individual_Game
  }

  while (length of _Individual.Action_History < Original_Action_Length) {
    set Chosen_Action to choose with equal Weights from
Individual_Game.Decision_Space

    execute Chosen_Action in Individual_Game
    append Chosen_Action to _Individual.Action_History
  }
}

```

```
function Fallback_Cut_And_Generation(_Generation, _Next_Generation,
_Processed_Indices, _Parameters) {
  foreach (_Individual in _Generation) {
    if (_Individual index inside _Generation in _Processed_Indices) {
      continue
    }

    set New_Individual to new Individual(value copy of
    _Individual.Action_History)
    Mutate_Cut_And_Generation(New_Individual, _Parameters, _Level_ID)

    append New_Individual to _Next_Generation
  }
}
```

De este método se extraen dos parámetros:

- *Random_Cut_Start_Percent*: Porcentaje que define el inicio del rango posible de la secuencia de acciones para el corte aleatorio, si su valor es distinto a -1 , entonces el método utilizará este modo.
 - *Last_Actions_To_Change*: Cantidad de acciones que eliminar del final de cada individuo para luego regenerar aleatoriamente, si su valor es distinto a 0, entonces el método utilizará este modo si es que no se definió que se usará el otro método previamente.
- Regeneración: Descarta a todos los individuos que no compartan ningún estado con algún otro individuo no emparejado, y por cada uno genera un nuevo individuo con la misma cantidad de acciones con las que inició la primera generación, como se mostró en la Figura 16.

```
function Fallback_Regeneration(_Next_Generation, _Parameters, _Level_ID) {
  set _Next_Generation to _Next_Generation joined with Create_Generation(new
  Generation_Creation_Parameters(_Parameters.Generation_Size - length of
  _Next_Generation, _Parameters.Actions_Amount), _Level_ID)
}
```

De este método no se extrae ningún parámetro nuevo, ya que este copia los parámetros de la creación de la primera generación.

- Clonación: Clona todos los individuos que no comparten ningún estado con algún otro individuo no emparejado para la siguiente generación, como se mostró en la Figura 17.

```
function Fallback_Clone(_Generation, _Next_Generation, _Processed_Indices) {  
  foreach (_Individual in _Generation) {  
    if (_Individual index inside _Generation is in _Processed_Indices) {  
      continue  
    }  
  
    append new Individual(value copy of _Individual.Action_History) to  
    _Next_Generation  
  }  
}
```

De este método no se extrae ningún parámetro.

- Mutación de los atributos de los individuos de la nueva generación: Se implementaron dos métodos, el segundo en implementarse, Corte y Generación, ya fue explicado en la definición del método de respaldo con el mismo nombre; el primero, Final Aleatorio, agrega N nuevas acciones válidas aleatorias a cada individuo al final de la secuencia de acciones de cada uno. De este método solo se extrae un parámetro:
 - *New_Actions_Amount*: Cantidad N de nuevas acciones válidas aleatorias que generar para cada individuo al final de su secuencia de acciones.

```
function Mutate_Random_Ending(_Individual, _Parameters, _Level_ID) {  
  set Individual_Game to new Game(_Level_ID)  
  
  foreach (_Action in _Individual.Action_History) {  
    execute _Action in Individual_Game  
  }  
  
  for (set _ to 0; _ < _Parameters.New_Actions_Amount; _++) {  
    set Chosen_Action to choose with equal Weights from  
Individual_Game.Decision_Space  
  
    execute Chosen_Action in Individual_Game  
    append Chosen_Action to _Individual.Action_History  
  }  
}
```

4.2 Resultados del Algoritmo Genético Configurado Manualmente

Para configurar el algoritmo genético, se realizaron pruebas a mano, modificando sus parámetros hasta encontrar resultados favorables (definido como lograr que el algoritmo encuentre alguna solución al nivel 3 en tres intentos consecutivos), y esos valores fueron usados como los parámetros nominales para el análisis de sensibilidad:

- *Generation_Size* = 100
- *Actions_Amount* = 10
- *Length_Weight* = 10
- *Obtained_Keys_Bonus* = 500
- *Used_Keys_Bonus* = 1.000
- *Has_Won_Bonus* = 1.000.000
- *Has_Looped_Penalty* = -1.000.000
- *Search_Start_Percent* = 0,2
- *Search_End_Percent* = 1
- *Fallback_Method* = { Method: "Bi-BFS", Parameters: { Max_Depth: 12 } }

- *Mutation_Method* = { Method: "Random Ending", Parameters: { New_Actions_Amount: 5 } }

Así, el algoritmo genético pudo resolver desde el nivel 1 hasta el 6, obteniendo las siguientes métricas promedio, calculadas solamente con los intentos exitosos que tuvo el algoritmo en cada nivel:

Nivel	Promedio de cantidad de acciones	Tiempo promedio (en segundos)
1	2	0,097
2	4	0,113
3	16,667	1,360
4	169,333	95,429
5	811,333	5.384,680
6	229,333	223,453

Tabla 6: Resultados del algoritmo genético con los parámetros definidos manualmente. El algoritmo no logró completar los niveles 7 y 8. (Fuente: Elaboración propia)

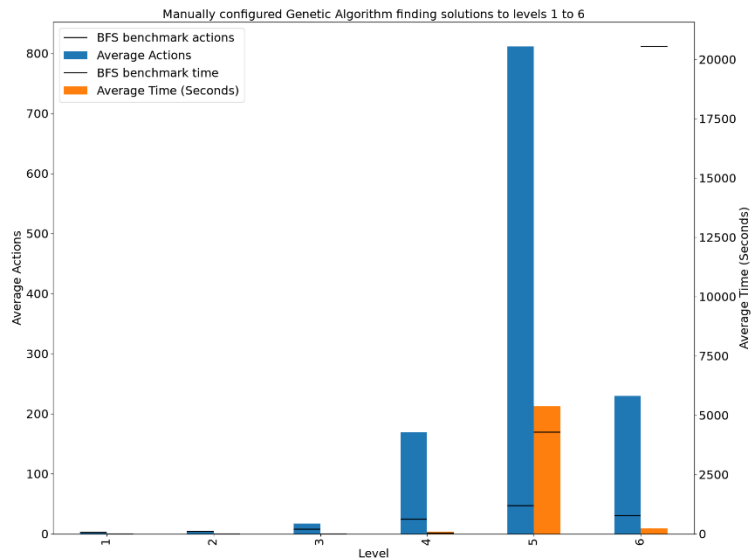


Gráfico 10: Visualización de la Tabla 6. (Fuente: Elaboración propia)

4.3 Análisis de Sensibilidad

4.3.1 Definición e Implementación

Para concretizar el alcance del análisis de sensibilidad, se determinó que para cada valor a probar se realizaría el análisis por factor único ejecutando cinco intentos, en únicamente el nivel 3, por cada combinación del método de respaldo utilizado en la fase de cruce y del método de mutación, resultando en 15 combinaciones de métodos posibles:

- BFS Bidireccional con Final Aleatorio
- BFS Bidireccional con Corte y Generación utilizando *Random_Cut_Start_Percent*
- BFS Bidireccional con Corte y Generación utilizando *Last_Actions_To_Change*
- Corte y Generación utilizando *Random_Cut_Start_Percent* con Final Aleatorio
- Corte y Generación utilizando *Random_Cut_Start_Percent* con Corte y Generación utilizando *Random_Cut_Start_Percent*
- Corte y Generación utilizando *Random_Cut_Start_Percent* con Corte y Generación utilizando *Last_Actions_To_Change*
- Corte y Generación utilizando *Last_Actions_To_Change* con Final Aleatorio
- Corte y Generación utilizando *Last_Actions_To_Change* con Corte y Generación utilizando *Random_Cut_Start_Percent*
- Corte y Generación utilizando *Last_Actions_To_Change* con Corte y Generación utilizando *Last_Actions_To_Change*
- Regeneración con Final Aleatorio
- Regeneración con Corte y Generación utilizando *Random_Cut_Start_Percent*
- Regeneración con Corte y Generación utilizando *Last_Actions_To_Change*
- Clonación con Final Aleatorio
- Clonación con Corte y Generación utilizando *Random_Cut_Start_Percent*
- Clonación con Corte y Generación utilizando *Last_Actions_To_Change*

Luego se determinaron los parámetros y valores de estos para probar en cada combinación, que fueron todos los parámetros (menos *Has_Won_Bonus* y *Has_Looped_Penalty*, que se mantuvieron en sus valores nominales), con los siguientes valores:

- *Generation_Size* = 2; 5; 10; 20; 35; 50; 75; 200
- *Actions_Amount* = 1; 3; 5; 15; 20; 30; 40; 50; 80; 100
- *Length_Weight* = 1; 5; 50; 100; 1.000
- *Obtained_Keys_Bonus* = 10; 50; 100; 200; 350; 650; 800; 1.000; 5.000; 10.000
- *Used_Keys_Bonus* = 10; 50; 100; 200; 350; 500; 800; 2.000; 5.000; 10.000
- *Search_Start_Percent* = 0.01; 0.05; 0.1; 0.15; 0.25; 0.35; 0.5; 0.6; 0.7; 0.8
- *Search_End_Percent* = 0.25; 0.3; 0.35; 0.4; 0.45; 0.5; 0.6; 0.7; 0.8; 0.9
- Si el método de respaldo de la fase de cruce es BFS Bidireccional:
 - *Max_Depth* = 1; 2; 3; 5; 8; 10; 15; 18
- Si el método de respaldo de la fase de cruce es Corte y Generación utilizando *Random_Cut_Start_Percent*:
 - *Random_Cut_Start_Percent* = 0.01; 0.05; 0.1; 0.15; 0.2; 0.4; 0.5; 0.6; 0.7; 0.8
- Si el método de respaldo de la fase de cruce es Corte y Generación utilizando *Last_Actions_To_Change*:
 - *Last_Actions_To_Change* = 1; 3; 5; 8; 15; 25; 35; 50; 80; 100
- Si el método de mutación es Final Aleatorio:
 - *New_Actions_Amount* = 1; 2; 3; 5; 8; 10; 20; 35; 50
- Si el método de mutación es Corte y Generación utilizando *Random_Cut_Start_Percent*:
 - *Random_Cut_Start_Percent* = 0.01; 0.05; 0.1; 0.15; 0.2; 0.4; 0.5; 0.6; 0.7; 0.8
- Si el método de mutación es Corte y Generación utilizando *Last_Actions_To_Change*:
 - *Last_Actions_To_Change* = 1; 3; 5; 8; 15; 25; 35; 50; 80; 100

4.3.2 Resultados

Una vez realizado el análisis de sensibilidad, se agruparon los resultados en gráficos según la combinación de métodos utilizados y el parámetro sensibilizado, resultando en gráficos como el siguiente, los demás gráficos para el nivel 3 se encuentran disponibles en el repositorio de la simulación del juego (A.2):

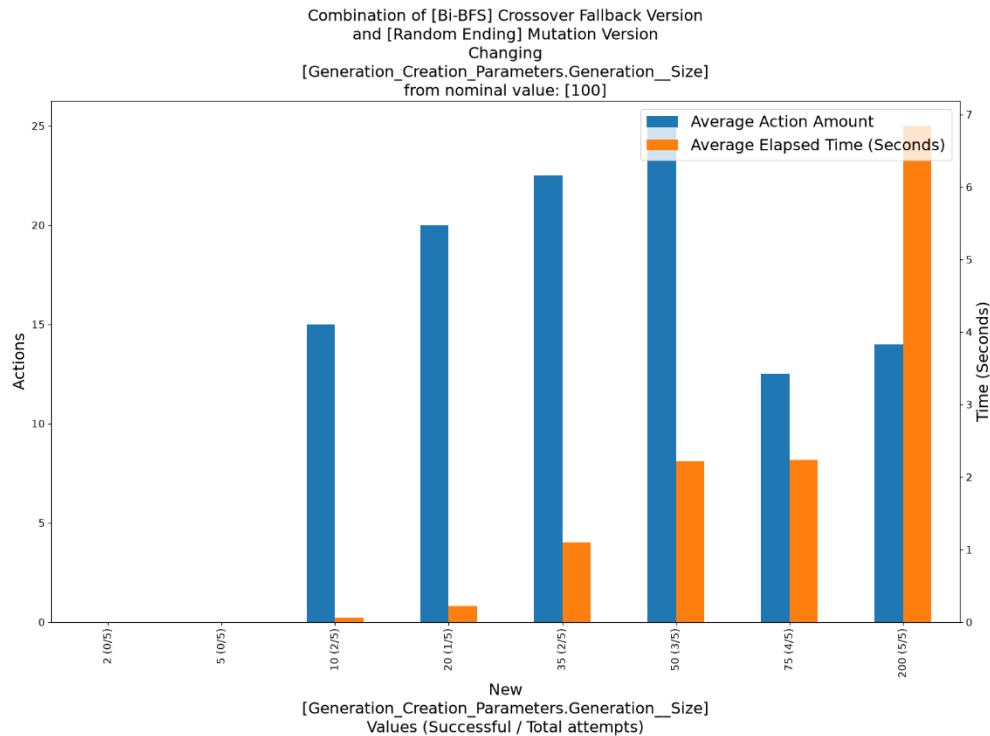


Gráfico 11: Ejemplo de resultado del análisis de sensibilidad, graficando la cantidad promedio de acciones y el tiempo promedio de ejecución de los intentos al sensibilizar el tamaño de la generación usando los métodos BFS Bidireccional y Final Aleatorio. (Fuente: Elaboración propia)

Luego se determinó el mejor valor para cada parámetro dentro de cada combinación de métodos, definiendo el mejor como el valor que tuviera a lo menos un 60% de éxito en completar el nivel 3, y que minimizara la cantidad promedio de acciones realizadas para superar el nivel, o el valor que minimizara el tiempo promedio de ejecución del algoritmo. Se encontraron estos dos mejores valores para cada parámetro y se agruparon según la combinación de métodos utilizados, resultando en gráficos como los siguientes:

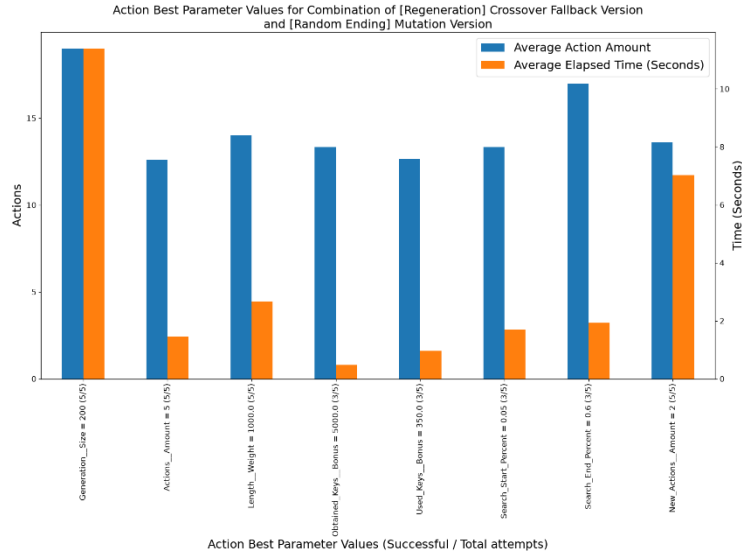


Gráfico 12: Ejemplo de resultado del análisis de sensibilidad, comparando los mejores valores en términos de cantidad promedio de acciones de cada parámetro que obtuvo al menos un resultado superando el mínimo del 60% de éxito dentro de la combinación de métodos Regeneración y Final Aleatorio. (Fuente: Elaboración propia)

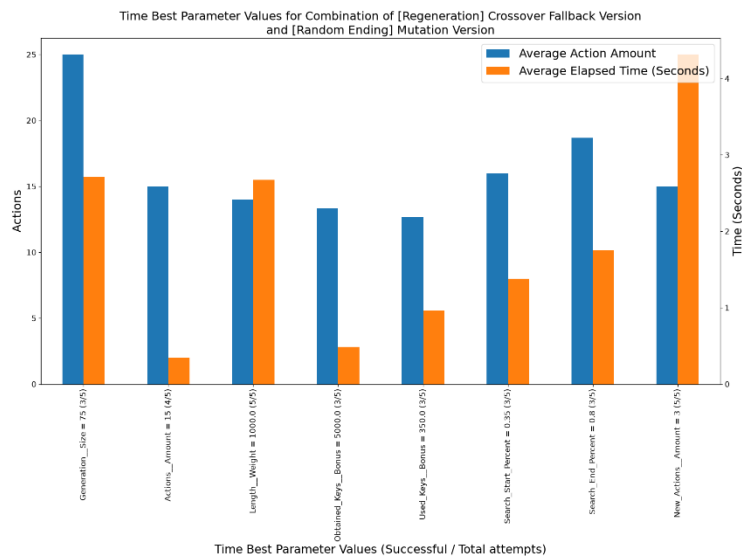


Gráfico 13: Ejemplo de resultado del análisis de sensibilidad, comparando los mejores valores en términos de tiempo promedio de ejecución de cada parámetro que obtuvo al menos un resultado superando el mínimo del 60% de éxito dentro de la combinación de métodos Regeneración y Final Aleatorio. (Fuente: Elaboración propia)

La siguiente agrupación se realizó con los mejores cambios para cada combinación de métodos, nuevamente según ambos criterios por separado: minimizar la cantidad promedio de acciones realizadas para completar el nivel 3, o minimizar el tiempo promedio de ejecución del algoritmo, resultando en los dos siguientes gráficos:

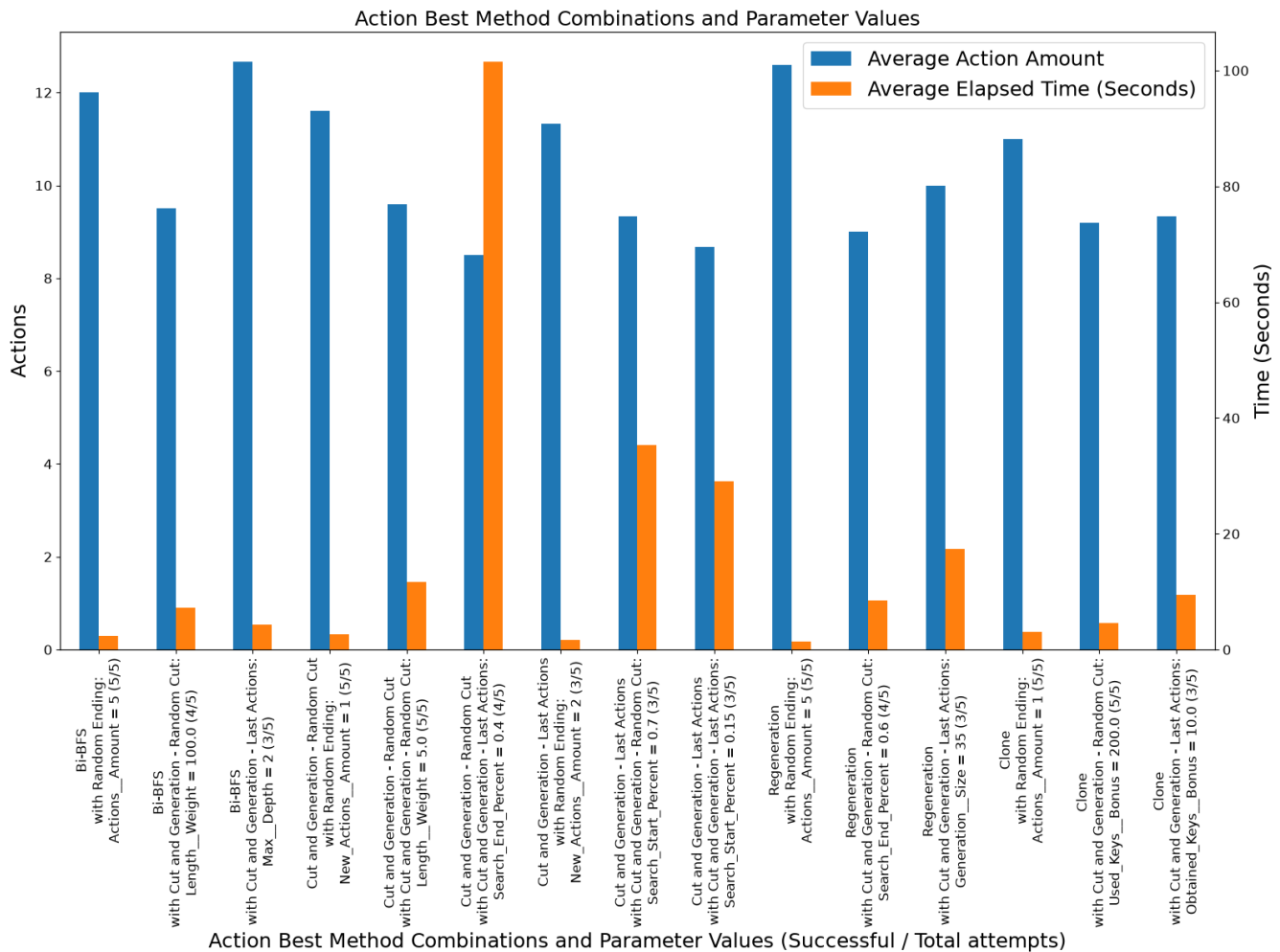


Gráfico 14: Resultado del análisis de sensibilidad, comparando los mejores cambios en términos de cantidad promedio de acciones de cada método. (Fuente: Elaboración propia)

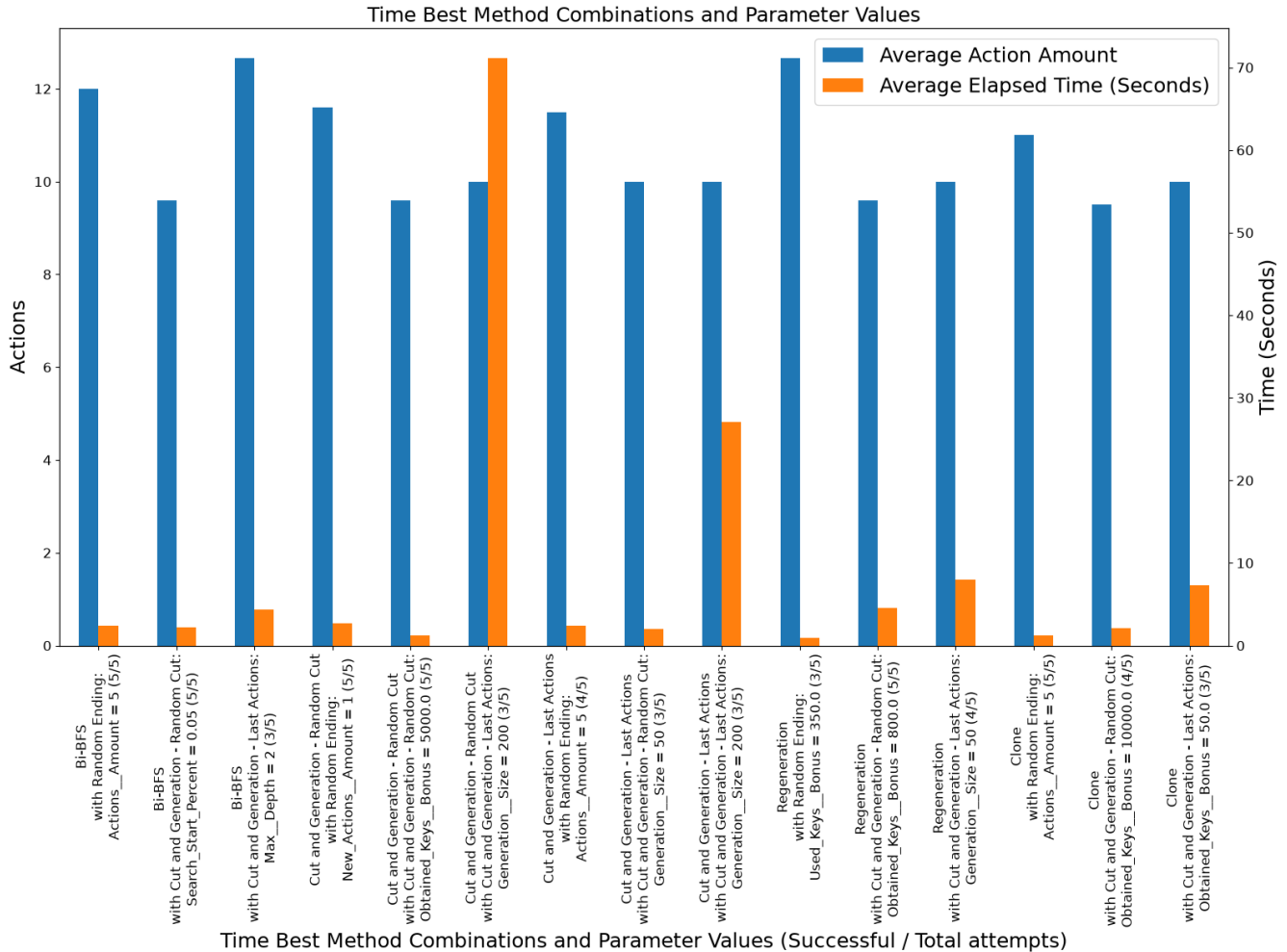


Gráfico 15: Resultado del análisis de sensibilidad, comparando los mejores cambios en términos de tiempo promedio de ejecución de cada método. (Fuente: Elaboración propia)

Finalmente, se determinó el mejor cambio y combinación de métodos para minimizar la cantidad promedio de acciones realizadas para completar el nivel 3, y el mejor cambio y combinación de métodos para minimizar el tiempo promedio de ejecución del algoritmo al intentar resolver el nivel 3, los que resultaron ser:

- Para minimizar la cantidad promedio de acciones, con un 80% de éxito: utilizando el método de respaldo de la fase de cruce Corte y Generación con *Random_Cut_Start_Percent* y el método de mutación Corte y Generación con *Last_Actions_To_Change*, se debe cambiar *Search_End_Percent* de su valor nominal de 1 a 0,4.

- Para minimizar el tiempo promedio de ejecución, con un 60% de éxito: utilizando el método de respaldo de la fase de cruce Regeneración y el método de mutación Final Aleatorio, se debe cambiar *Used_Keys_Bonus* de su valor nominal de 1.000 a 350.

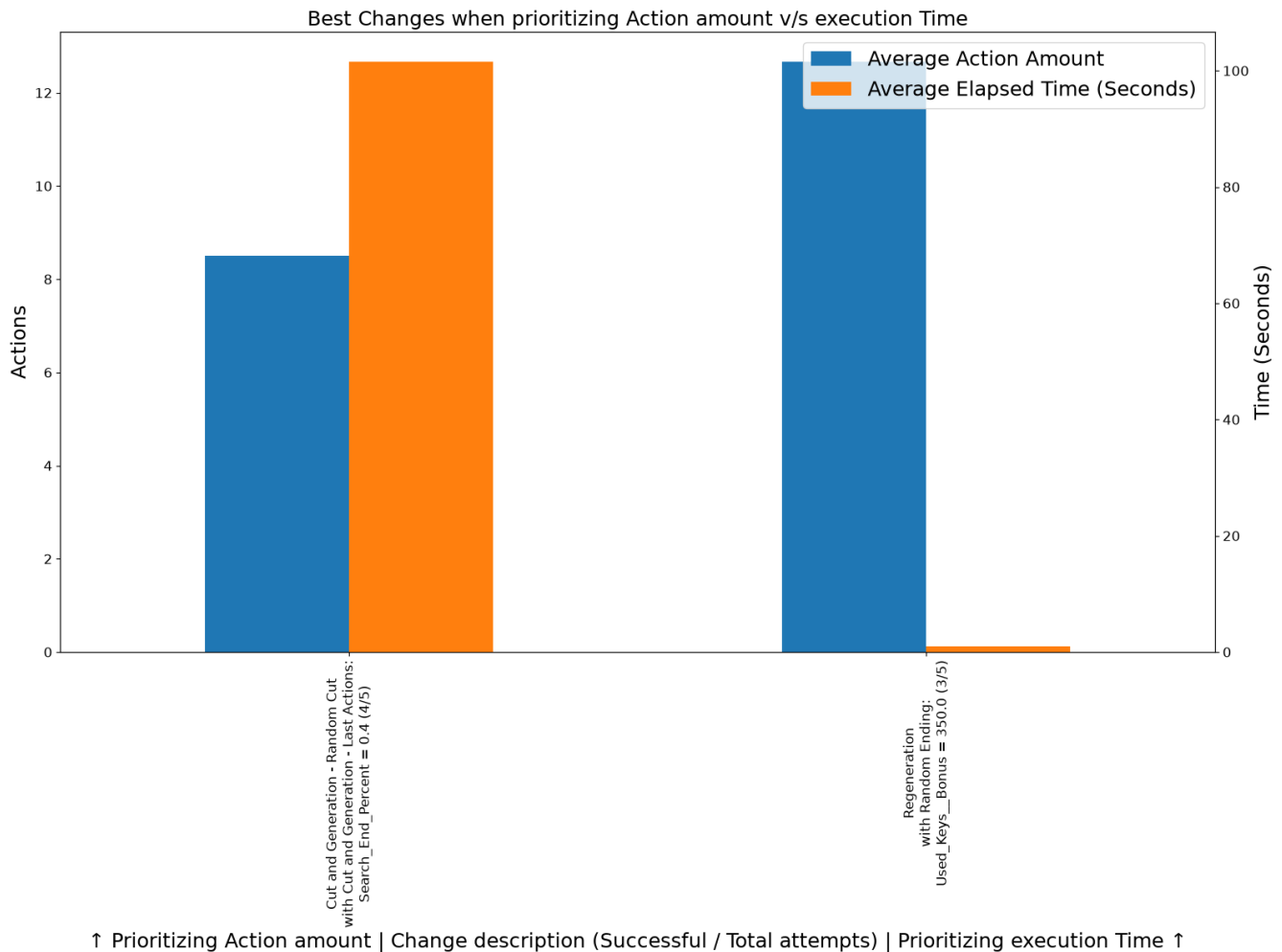


Gráfico 16: Resultado final del análisis de sensibilidad, concluyendo sobre cuál combinación de métodos usar y qué parámetro alterar en estas para minimizar la métrica deseada. (Fuente: Elaboración propia)

4.4 Resultados del Algoritmo Genético Modificado

Una vez modificado el algoritmo genético con ambos cambios sugeridos por la sección anterior por separado, se ejecutó nuevamente el algoritmo genético, logrando superar en el caso con la modificación orientada a minimizar la cantidad de acciones realizadas para completar el nivel los niveles 1, 2 y 3; y en el caso con la modificación orientada a minimizar el tiempo promedio de ejecución del algoritmo, este logró completar los niveles desde el 1 hasta el 6; con las siguientes métricas para cada caso:

Nivel	Promedio de cantidad de acciones	Tiempo promedio (en segundos)
1	10	0,098
2	10	0,132
3	9,333	62,679

Tabla 7: Resultados del algoritmo genético con el cambio concluido para minimizar la cantidad promedio de acciones. El algoritmo no logró completar el nivel 4 ni ninguno posterior a este. (Fuente: Elaboración propia)

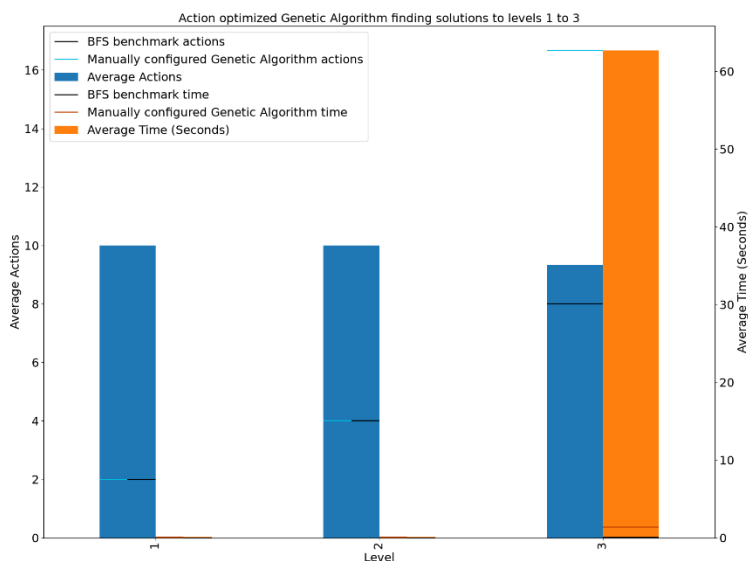


Gráfico 17: Visualización de la Tabla 7. (Fuente: Elaboración propia)

Nivel	Promedio de cantidad de acciones	Tiempo promedio (en segundos)
1	10	0,092
2	10	0,113
3	19,783	6,953
4	174,542	49,677
5	336,250	106,515
6	67,500	27,322

Tabla 8: Resultados del algoritmo genético con el cambio concluido para minimizar el tiempo promedio de ejecución. El algoritmo no logró completar el nivel 7 ni el nivel 8. (Fuente: Elaboración propia)

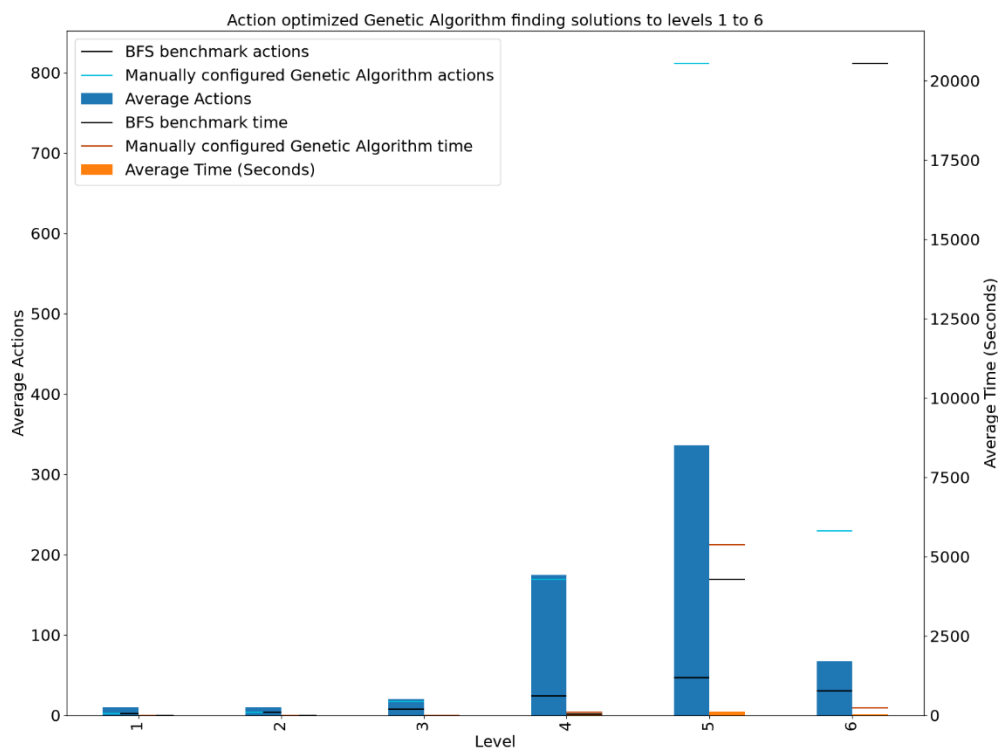


Gráfico 18: Visualización de la Tabla 8. (Fuente: Elaboración propia)

5. Discusión

Dados los resultados, se eligió como mejor solución optimizar el algoritmo genético según el criterio de minimizar el tiempo promedio de ejecución del algoritmo, que implica utilizar el método de respaldo para la fase de cruce Regeneración y el método de mutación Final Aleatorio, modificando *Used_Keys_Bonus* de su valor nominal de 1.000 a 350, ya que se prefirió obtener alguna solución posible en el menor tiempo posible, que intentar obtener alguna solución óptima con el algoritmo genético, que en la mayoría de casos no logra hacer. Con esta solución la investigación se acercó a cumplir el objetivo de encontrar o diseñar un algoritmo capaz de determinar rápidamente si un nivel dado tiene solución o no, sin embargo, el algoritmo genético no cumplió ese objetivo completamente, ya que no logró encontrar ninguna solución para los niveles 7 y 8, y no obtuvo un 100% de éxito en todos los demás niveles que sí logró completar. Por otra parte, la investigación obtuvo resultados casi completamente satisfactorios en cuanto a los *benchmarks* realizados con DFS y BFS, pues se descubrieron soluciones óptimas y la mínima cantidad de acciones necesarias para completar cada nivel a excepción del último, además, el algoritmo genético tuvo éxito al encontrar varias soluciones novedosas para los niveles 4, 5 y 6, lo que parcialmente satisface el objetivo de buscar nuevas soluciones y tipos de estas para todos los niveles.

Si bien el algoritmo genético logró completar todos los niveles salvo los últimos dos, este no tuvo un 100% de éxito en todos sus intentos, y este porcentaje disminuyó significativamente a medida que el algoritmo genético se enfrentaba a un siguiente nivel. Lo anterior puede deberse a alguna(s) de las siguientes tres razones:

- 1) Los niveles más avanzados tienen más acciones posibles en cada estado: esto se puede ver reflejado en el tamaño de los espacios de búsqueda de los niveles mostrados en las Tablas 4 y 5, y en los Gráficos 4 al 9, en los cuales es observable un crecimiento exponencial para la cantidad de estados únicos que tiene un nivel a medida que aumenta la cantidad de acciones realizadas en este. Además, ignorando los dos primeros niveles por resolverse trivialmente, el tercer nivel permite un máximo de hasta 6 acciones posibles en algunos de sus estados, al igual que el nivel 4, sin embargo, esto crece considerablemente para los siguientes niveles: para el nivel 5, pueden existir hasta 9 acciones posibles en algunos de sus estados, en el nivel 6 pueden existir hasta 7 acciones posibles, en el nivel 7 pueden existir hasta 9 acciones posibles, y para el nivel 8, vuelven a poder existir hasta 6 acciones posibles en algunos de sus estados; por lo que, a mayor cantidad de acciones posibles

dentro de un nivel, mayor puede ser la desviación de los individuos de las generaciones del algoritmo genético, y por ende, mayor puede ser la probabilidad de que el algoritmo falle en encontrar alguna solución dentro del límite de generaciones utilizado.

- 2) Los niveles más avanzados son más largos: como se mostró en la Tabla 3 y Gráfico 3, en la mayoría de casos, cada nivel requiere un mínimo de acciones para poder completarse mayor que el nivel anterior, y junto con lo explicado en la posibilidad anterior respecto a las Tablas 3 y 4, y a los Gráficos 4 al 9, al tener más acciones mínimas que realizar para completar cada nivel, sumado a que con cada acción realizada existan cada vez más estados posibles que antes, aumentan las probabilidades de que los individuos de las generaciones del algoritmo genético se desvíen de una posible ruta que los llevaría a completar el nivel.
- 3) Los niveles más avanzados requieren de ciertas secuencias de acciones o estados específicos: por ejemplo, el nivel 8 requiere de una secuencia de estados particular al final del nivel para poder alcanzar la salida; y de manera similar, los niveles 5, 6 y 7 requieren de algún u otro tipo de secuencia de acciones o estados particular para poder avanzar a otras zonas del nivel, obtener algunas llaves específicas, y/o completar el nivel; esto causa que la exploración que realicen los individuos de las generaciones del algoritmo genético tengan que encontrar esas secuencias y ejecutarlas completamente para superar los últimos niveles, que se vuelven “cuellos de botella” en los que convergen todos los posibles estados previos a estas secuencias, y que aumenta la probabilidad de que el algoritmo no logre completar esos niveles antes de alcanzar el límite de generaciones utilizado.

5.1 Proyecciones a Futuro

A futuro, para la continuación de la presente investigación se espera investigar desde una perspectiva de modelamiento matemático junto con el programático, para descubrir más propiedades del juego; también se creará una mejor interfaz que pueda visualizar las simulaciones de la misma forma que se visualiza el juego en estado normal. Para mejorar el rendimiento de la simulación, se considerará reescribir la simulación para permitir *multi-threading*, y en el caso de permanecer en el lenguaje de programación Python, se considerará utilizar Taichi (Taichi Lang: High-performance Parallel Programming In Python, s. f.) o Cython (Cython: C-Extensions For Python, s. f.) para reducir el tiempo

de ejecución de la simulación y así poder permitir, por ejemplo: obtener los tamaños de los espacios de búsqueda de los niveles a mayores profundidades, y quizás obtener el espacio de búsqueda completo de todos los niveles; obtener alguna solución óptima del nivel 8 utilizando BFS; aumentar considerablemente el tamaño de las generaciones del algoritmo genético, para averiguar si con una cantidad significativamente mayor de individuos el algoritmo puede resolver más consistentemente los niveles que ya puede completar, y/o superar los niveles 7 y 8, que aún no logra completar. Dentro de las consideraciones de optimización para la simulación del juego, se considerará migrar la simulación a C++ para aprovechar la mejora en rendimiento que eso significará, junto con las optimizaciones antes mencionadas si es que aún aplican para el lenguaje.

5.1.1 Otros Algoritmos y Métodos

Dado que la presente investigación e informe consideraron solamente el uso de DFS, BFS y el algoritmo genético, otro aspecto a explorar en el futuro de la investigación es expandir los métodos posibles para el algoritmo genético, y utilizar otros algoritmos y métodos que podrían ser prometedores para cumplir los objetivos planteados para la investigación del juego. Para expandir los métodos del algoritmo genético se considerarán:

- Un nuevo método de evaluación que evalúe a los individuos según “buenas prácticas” que se deberán definir, como mantener la mayor (o menor) cantidad de acciones posibles en cada estado que se visite, u otras prácticas que dependerán de otras características de los niveles, como su distribución en la cuadrícula, y su forma.
- Un nuevo método de evaluación basado en novedad (Doucette & Heywood, 2010) de estados, ya sea para cada individuo por separado, dentro de cada generación, o de manera general para toda la ejecución del algoritmo genético.
- Encadenar métodos existentes, permitiendo que se altere el orden, y utilizando parámetros para todos los métodos en las cadenas.

Respecto a utilizar otros algoritmos o métodos que podrían ser prometedores, se planean considerar:

- Aprendizaje de políticas (Liu et al., 2020), ya que podría ayudar a analizar niveles en profundidad y obtener información de patrones propios de cada nivel.

- Aprendizaje por refuerzo (Sutton & Barto, 2018), ya que puede ser un método con el que se pueden explorar comportamientos diferentes de jugadores, y así obtener de cada nivel cómo estos podrían enfrentarse a los niveles del juego.

El método de búsqueda de árboles de Montecarlo (Abramson, 1987) es también una consideración, aunque no se esperan resultados favorables de este debido a la dudosa factibilidad de que se puedan terminar todos los *playouts* en un tiempo razonable, además, como método solo se espera que pueda ayudar a optimizar soluciones ya encontradas en cada nivel por separado en lugar de encontrar soluciones de manera independiente, o generalizar a cualquier nivel, lo que se acerca parcialmente a un paso siguiente en la investigación, pero no al más próximo.

Finalmente, se considerará la factibilidad de algunas propuestas de algoritmos originales, y se probarán en caso de determinarse como prometedores para poder satisfacer alguno de los objetivos de la investigación. Las propuestas de algoritmos son las siguientes:

- BFS basado en eventos: este algoritmo consiste en ejecutar el algoritmo BFS desde el estado inicial, y ejecutar nuevas instancias del algoritmo en paralelo cuando este o alguna de las instancias realice una acción de evento, que se definen para el caso del juego como acciones que hagan que el jugador obtenga o utilice una llave. La idea detrás de esta propuesta es crear “puntos de control” (o *checkpoints*) en la búsqueda, realizando nuevas búsquedas centradas en los estados inmediatamente posteriores a esas acciones de evento, compartiendo entre todas las instancias el conjunto de estados visitados, y así realizar la búsqueda de algún estado terminal desde estados que a través de todos los niveles estarán más cercanos a completar estos que los estados iniciales. No se espera que este algoritmo, de ser implementado, obtenga soluciones óptimas para los niveles, pero sí que obtenga varias soluciones diferentes en poco tiempo.
- Ramas de BFS: este algoritmo consiste en generar ramas de exploración del espacio de búsqueda de un nivel del juego, similares a los individuos de la primera generación del algoritmo genético, pero asegurando de alguna manera que estos no compartan ningún estado entre las otras ramas, no contengan ciclos, y por sobre todo, que estén distribuidas lo más uniformemente posible en todo el espacio de búsqueda del nivel del juego; para luego elegir estados particulares en cada una de las ramas, manteniendo la máxima distancia posible entre los estados seleccionados, y ejecutar instancias del algoritmo BFS en cada uno de esos estados en paralelo, compartiendo entre todas las instancias el conjunto de estados visitados. La idea detrás de este algoritmo es generar más

estados “iniciales” para minimizar la profundidad que explora cada instancia de BFS, pero sujeto a la mínima cantidad de estados seleccionados como estados iniciales adicionales, de tal manera que, supuestamente, cada posible estado del nivel está lo suficientemente cerca de algún estado inicial como para ser explorado por alguna instancia de BFS en poco tiempo, para así encontrar alguna solución al nivel lo más rápido posible, reconstruyendo la secuencia de acciones como la secuencia de acciones realizada por la rama hasta llegar al estado “inicial”, unido a la secuencia de acciones que obtenga la instancia de BFS que alcance un estado terminal. Esta propuesta de algoritmo requiere de una mejor definición de sus partes, por lo que para evaluar su factibilidad de satisfacer alguno de los objetivos de la investigación, primero se desarrollará en más detalle la definición y características del algoritmo.

6. Bibliografía

- 1) 2swap. (2025, 22 agosto). I solved Klotski [Video]. YouTube. <https://www.youtube.com/watch?v=YGLNyHd2w10>
- 2) Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms, fourth edition. MIT Press.
- 3) Gendreau, M., & Potvin, J. (2010). Handbook of Metaheuristics. En International series in management science/operations research/International series in operations research & management science. <https://doi.org/10.1007/978-1-4419-1665-5>
- 4) Saltelli, A., Ratto, M., Andres, T., Campolongo, F., Cariboni, J., Gatelli, D., Saisana, M., & Tarantola, S. (2007). Global Sensitivity Analysis. The primer. <https://doi.org/10.1002/9780470725184>
- 5) Bozorg-Haddad, O., Loáiciga, H., & Zolghadr-Asli, B. (2021). A Handbook on Multi-Attribute Decision-Making Methods. <https://doi.org/10.1002/9781119563501>
- 6) Taichi Lang: High-performance Parallel Programming in Python. (s. f.). <https://www.taichi-lang.org/>
- 7) Cython: C-Extensions for Python. (s. f.). <https://cython.org/>
- 8) Doucette, J., & Heywood, M. I. (2010). Novelty-Based Fitness: An Evaluation under the Santa Fe Trail. En Lecture notes in computer science (pp. 50-61). https://doi.org/10.1007/978-3-642-12148-7_5
- 9) Liu, Y., Yang, J., Chen, L., Guo, T., & Jiang, Y. (2020). Overview of Reinforcement Learning Based on Value and Policy. IEEE, 598-603. <https://doi.org/10.1109/ccdc49329.2020.9164615>
- 10) Sutton, R. S., y Barto, A. G. (2018). Reinforcement learning: An introduction (2.^a ed.). The MIT Press. <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- 11) Abramson, Bruce (1987). The Expected-Outcome Model of Two-Player Games. Technical report, Department of Computer Science, Columbia University. http://academiccommons.columbia.edu/download/fedora_content/download/ac:142327/CONTENT/CU-CS-315-87.pdf

7. Anexos

A.1) Demo pública del juego en desarrollo, jugable directamente desde el navegador: <https://ai-0.itch.io/project-know-the-space-i>

A.2) Repositorio con la simulación del juego escrita en Python, donde también se pueden encontrar todos los gráficos del análisis de sensibilidad: <https://github.com/RobTamGon/Simulation-KnowTheSpace>